

From Source to Execution: Design and Implementation of a Statically-Typed Programming Language with Type Inference

by *The Token Trio*

Aye Khin Khin Hpone (Yolanda Lim), st125970

Applegate T. Tun Oo, st126690

Win Htut Naing, st126687

AT70.07 — Programming Languages and Compilers

Assignment 2

Instructor: Prof. Phan Minh Dung

Teaching Assistant: Akkradet Sinsamersuk

Asian Institute of Technology

School of Engineering and Technology

Thailand

April 2026

ABSTRACT

This report presents the design and implementation of a statically-typed interpreted programming language realised through a four-stage compiler pipeline. The language provides four primitive types — Integer, Float, Boolean, and String — and supports arithmetic and equality expressions, variable assignment with type inference from first use, conditional and iterative control flow, user-defined functions with value parameter passing, and a built-in callable `print()` function. Types are inferred from program context rather than declared explicitly, combining static type safety with a declaration-free syntax. The pipeline consists of a hand-written lexer, a recursive-descent parser that constructs an abstract syntax tree together with an inspectable parse-tree view, a static type checker that performs inference and rejects ill-typed programs before execution, and a tree-walking interpreter. The type checker implements a syntax-directed type inference algorithm over the abstract syntax tree, enforcing static type safety without explicit declarations and rejecting ill-typed programs before execution. Both a command-line runner and an interactive desktop interface expose each pipeline stage for inspection. The report documents the formal grammar, the type inference algorithm, the system architecture, and the automated test suite of 75 tests that validates the complete implementation.

CONTENTS

ABSTRACT	ii
1 INTRODUCTION	1
2 LANGUAGE DESIGN AND GRAMMAR	3
2.1 Types	3
2.1.1 Integer	3
2.1.2 Float	3
2.1.3 Boolean	4
2.1.4 String	4
2.1.5 Void	4
2.2 Static Typing	4
2.3 Token Specification	5
2.4 Context-Free Grammar	5
2.4.1 Program Structure	5
2.4.2 Expressions	6
2.4.3 Boolean Expressions	6
2.4.4 Statements	7
2.4.5 Function Definitions and Calls	7
2.5 Operator Precedence and Associativity	8
2.6 Design Decisions: No Overloading	9
3 LEXICAL ANALYSIS AND SYMBOL TABLE	11
3.1 Token representation	11
3.2 Lexer implementation	12

3.3	Symbol table and semantic structure	14
3.3.1	Example symbol table hierarchy	15
3.4	Role of the symbol table in the pipeline	16
4	RECURSIVE-DESCENT PARSING, PARSE TREE, AND ABSTRACT SYNTAX TREE	18
4.1	Abstract syntax tree representation	18
4.1.1	Expressions	19
4.1.2	Statements	19
4.2	Parser implementation	20
4.2.1	Rationale for recursive-descent parsing	20
4.3	Parse-tree view	21
4.3.1	Statement parsing	22
4.3.2	Expression parsing and precedence	22
4.3.3	Helper functions and lookahead	24
4.4	AST printing	24
4.5	Parser and type checker boundary	24
5	TYPE INFERENCE ALGORITHM	25
5.1	Overview	25
5.2	Type Environment	25
5.3	Type Inference Procedure	25
5.3.1	Example type inference trace	27
5.4	Inference Rules	27
5.4.1	Literals	27
5.4.2	No Operator Overloading	28
5.4.3	Arithmetic Expressions	28
5.4.4	Boolean Expressions	29

5.4.5	Assignment Statement	30
5.4.6	If-Then-Else	30
5.4.7	While-Loop	30
5.4.8	Function Definition and Call	30
5.5	Type Error Reporting	31
5.6	Example Type Inference Traces	32
6	SYSTEM ARCHITECTURE AND IMPLEMENTATION	33
6.1	Compiler pipeline	33
6.1.1	End-to-end pipeline walkthrough	34
6.1.2	Entry points and shared pipeline	38
6.2	Project structure	39
6.3	Technology stack	39
6.4	Type checker	40
6.4.1	Scope of the implemented type system	40
6.5	Interpreter	40
6.5.1	Scope of the implemented runtime	41
6.6	Graphical user interface	41
6.6.1	Screenshots	42
6.7	Command-line interface	43
7	TESTING AND VERIFICATION	45
7.1	Manual Testing	45
7.1.1	Lexer and Symbol Table	45
7.1.2	Parser, Parse Tree, and AST	46
7.1.3	Arithmetic Expressions	47
7.1.4	Boolean Expressions	47
7.1.5	Assignment Statements	48

7.1.6	If-Then-Else	48
7.1.7	While-Loop	48
7.1.8	Functions	48
7.1.9	Print	48
7.1.10	Unary Minus	48
7.2	Type Error Detection	49
7.3	Automated Test Suite	49
7.4	Error Handling	51
7.5	Limitations and Future Work	51
8	CONCLUSION	52
	REFERENCES	53
	APPENDIX A: SOURCE CODE	54
	APPENDIX B: TEAM MEMBERS AND CONTRIBUTIONS	54

CHAPTER 1

INTRODUCTION

Programming languages define the formal contract between a programmer’s intent and machine execution. Building a language processor from source text through to evaluated output requires a coherent pipeline of components, each of which must correctly transform its input before passing it downstream. Lexical analysis identifies atomic tokens; parsing imposes hierarchical structure; static type checking validates semantic correctness before execution; and interpretation evaluates the validated program. When these stages are connected cleanly, errors are caught at the earliest possible point, and the overall system remains tractable to reason about, test, and extend.

This report presents the design and implementation of a statically-typed interpreted language that realises this pipeline in full. The language supports four primitive types — Integer, Float, Boolean, and String — and provides arithmetic and equality expressions, assignment, conditional and iterative control flow, user-defined functions with value parameter passing, and a built-in callable `print()` function. Types are inferred from program context rather than declared explicitly, combining the safety of static typing with the conciseness of a declaration-free syntax. The implementation spans three principal components developed in pipeline order: the lexer and symbol table, the recursive-descent parser together with parse-tree and abstract-syntax-tree views, and the type checker with interpreter. Each component was completed before the next was begun, since each stage consumes the structured output of its predecessor.

The implemented system can be invoked from the command line or through an interactive desktop interface, both of which surface the five pipeline stages — tokens, parse tree, AST, inferred type table, and execution output — for inspection. The parse tree is provided as a grammar-oriented view for submission and debugging, while the AST remains the executable intermediate representation consumed by the type checker and interpreter. An automated regression suite of 75 tests verifies correctness across all stages and language features.

The main contributions of this project are:

- Design of a statically-typed programming language supporting Integer, Float, Boolean, and

String types without operator overloading.

- Implementation of a hand-written lexer and a recursive-descent parser with explicit parse-tree and abstract syntax tree generation.
- Development of a syntax-directed type inference and static type-checking system without explicit type declarations.
- Separation of integer and floating-point arithmetic operators following the Caml Light convention to eliminate semantic ambiguity.
- Construction of a complete end-to-end execution pipeline consisting of lexical analysis, parsing, semantic analysis, and interpretation.
- Development of both command-line and graphical interfaces exposing all compiler pipeline stages for inspection and debugging.
- Validation of the implementation through automated regression testing and semantic error verification.

The remainder of this report is organised as follows. Chapter 2 presents the language design, token specification, grammar, and operator precedence rules. Chapters 3 and 4 cover the front-end implementation in pipeline order: lexical analysis and the symbol table, then recursive-descent parsing and the AST. Chapter 5 explains the type inference algorithm and the semantic rules used by the checker. Chapter 6 presents the system architecture and implementation, including the end-to-end pipeline, project layout, and technology stack. Chapter 7 discusses testing and verification, and Chapter 8 concludes the report.

CHAPTER 2

LANGUAGE DESIGN AND GRAMMAR

2.1 Types

The language provides four primitive types, chosen to cover the core domains of numeric computation, logical control flow, and text manipulation while keeping the type system tractable and the semantics well-defined.

2.1.1 *Integer*

The Integer type represents whole numbers such as 0, 7, and 42. Integer literals are written as one or more decimal digits without quotation marks or a decimal point.

2.1.2 *Float*

The Float type represents real-valued numbers written with a decimal point, such as 3.14 or 0.5. Float values are used exclusively with the dot-suffixed arithmetic operators ($+. , -. , *. , /. ,$), following the Caml Light convention. Integer division ($/$) performs floor division and returns Integer: for example, $10 / 3$ evaluates to 3 : Integer.

The language intentionally does not perform implicit numeric promotion between Integer and Float values. This design decision follows directly from the assignment specification, which requires “no overloading with the usual precedence of multiplication and division over addition and subtraction.” In a language where a single $+$ operator accepts both $(Integer, Integer) \rightarrow Integer$ and $(Integer, Float) \rightarrow Float$, the operator effectively carries multiple type signatures — the compiler must inspect both operand types to determine the result type. This constitutes a form of operator overloading, even when described as “numeric promotion” or “widening coercion.” To eliminate all such ambiguity, the language follows the Caml Light convention of using separate operator tokens for integer and floating-point arithmetic: $+$, $-$, $*$, $/$ operate exclusively on Integer operands, while $+. , -. , *. , /. ,$ operate exclusively on Float operands. Mixed-type arithmetic such as $1 + 2.5$ is rejected by the type checker. Under this design, every operator token uniquely determines its operand types and result type — the compiler never needs to inspect operand types to resolve operator meaning, which is the defining property of a

language without operator overloading.

2.1.3 *Boolean*

The Boolean type represents logical truth values. The two Boolean literals are `true` and `false`. Boolean values are used in conditions and as results of comparison expressions.

2.1.4 *String*

The String type represents sequences of characters enclosed in double quotes, for example `"hello"`. Strings can be assigned to variables and printed through the built-in `print()` function. The surrounding quotes are stripped by the parser when the `Literal` node is created, so the value stored internally is the bare character sequence without surrounding quotes. At runtime, the built-in `print()` function re-adds double quotes around string values, so `print("plc")` outputs `"plc" : String`.

2.1.5 *Void*

Although the language exposes exactly four programmer-visible types, the implementation introduces a fifth value, `Void`, within the `LanguageType` enumeration. Its sole purpose is to provide a well-defined return type for functions that contain no `return` statement, allowing the symbol table to record a consistent entry for every function regardless of whether it produces a value. `Void` cannot appear in source code and is never visible to the programmer; it is a bookkeeping device used internally by the type checker. Therefore, `Void` does not extend the programmer-visible type system required by the assignment.

2.2 Static Typing

The language employs static typing, meaning that type errors are detected at type-checking time rather than at runtime. No explicit variable type declarations are required. Instead, the type checker infers types from literals, assignments, operations, function calls, and return statements. Once a variable type is established, later assignments must remain consistent with that type. For example, the following program is rejected at the type-checking stage before any execution occurs:

```
1 x = 5;  
2 x = "hello";
```

The type checker rejects the second assignment with:

```
1 [line 2, col 1] TypeError: Cannot assign String to variable 'x' of type
   Integer
```

This contrasts with dynamically-typed languages, where the same program would run without error until (or unless) a type-dependent operation was attempted.

2.3 Token Specification

Table 2.1 summarises the major token classes used by the language.

Table 2.1. Token Specification

Token Category	Example Lexemes
INT_LITERAL	0, 10, 42
FLOAT_LITERAL	3.14, 20.5
BOOL_LITERAL	true, false
STRING_LITERAL	"hello"
IDENTIFIER	variable and function names such as x, add
Arithmetic operators (integer)	+, -, *, /
Float arithmetic operators	+, -, *, / .
Comparison operators	==, !=
Assignment operator	=
Keywords	if, else, while, def, return
Delimiters	() { } , ;

2.4 Context-Free Grammar

The language grammar can be described by the context-free grammar $G = (V_N, V_T, P, S)$, where V_N is the set of non-terminals, V_T is the set of terminals, P is the set of production rules, and S is the start symbol program. The grammar is implemented through recursive-descent parsing functions.

2.4.1 Program Structure

```
1 program    -> statement* EOF
2 statement -> assignment
3           | if_stmt
4           | while_stmt
5           | func_def
6           | return_stmt
7           | expr ';' ;
```

2.4.2 Expressions

```
1 expr      -> comparison | additive
2 comparison -> additive ('==' | '!=') additive
3 additive  -> term (('+' | '-' | '+.' | '-.') term)*
4 term      -> factor (('*' | '/' | '*.' | '/.') factor)*
5 factor    -> '-' factor      (* integer unary negation *)
6           | '-.' factor     (* float unary negation *)
7           | INT_LITERAL
8           | FLOAT_LITERAL
9           | STRING_LITERAL
10          | BOOL_LITERAL
11          | IDENTIFIER
12          | IDENTIFIER '(' args? ')',
13          | '(' expr ')'
```

The comparison non-terminal is introduced so that the parse-tree renderer can label equality and inequality expressions explicitly. When an `expr` contains no `==` or `!=`, it expands directly to an additive; otherwise, it expands to a comparison node whose two operands are themselves additive sub-trees. Comparisons remain non-associative (one comparison per expression) and produce a Boolean result.

The operators `+`, `-`, `*`, and `/` operate exclusively on Integer operands. The dot-suffixed operators `+`, `-`, `*`, and `/` operate exclusively on Float operands. This follows the Caml Light convention of using distinct operator tokens for integer and float arithmetic, eliminating the need for implicit numeric promotion. Unary negation is similarly split: `-x` negates an Integer (desugared to `0 - x`), and `-.x` negates a Float (desugared to `0.0 -. x`).

Integer division and division by zero. `/` is floor (integer) division: both operands must be Integer, the result is Integer, and the value is $\lfloor a/b \rfloor$. For example, `7 / 2` evaluates to `3 : Integer`. Float division uses the distinct operator `/.` and requires two Float operands. Division by zero (`x / 0` or `x /. 0.0`) is detected at runtime and raises a `RuntimeError`.

2.4.3 Boolean Expressions

Comparison operators `==` and `!=` are the lowest-precedence binary operators and produce Boolean values. Because they are part of the general `expr` rule, Boolean expressions may appear both as control-flow conditions and as ordinary expressions. Their inferred type is Boolean, allowing comparison results to be assigned to variables, passed as arguments, or printed:

```
1 flag = (x == y);
2 print(a != b);
```

Here `flag` is inferred as `Boolean` from the assignment, and `print(a != b)` outputs `true` : `Boolean` or `false` : `Boolean`.

Strict arithmetic operands. The specification states that `==` and `!=` operate between two arithmetic expressions. We interpret “arithmetic” strictly as `Integer` and `Float`, consistent with the no-overloading rule. Both operands must also share the same numeric type (no implicit coercion). Therefore `Integer == Integer` and `Float == Float` are well-typed; `Integer == Float`, `Boolean == Boolean`, and `String == String` are all rejected at type-check time.

2.4.4 Statements

```
1 assignment  -> IDENTIFIER '=' expr ';'
2 if_stmt     -> 'if' '(' expr ')' block ('else' block)?
3 while_stmt  -> 'while' '(' expr ')' block
4 return_stmt -> 'return' expr ';'
5 block       -> '{' statement* '}'
```

Design note: print as a built-in callable

`print` is implemented as a normal function-call expression rather than as a dedicated statement form. Lexically, `print` is tokenised as an `IDENTIFIER`; syntactically, `print(x)` is parsed through the same `FunctionCall` path used for user-defined calls. Semantically it is a unary built-in of type $T \rightarrow ()$: it accepts one argument of any programmer-visible type, emits `value` : `Type` immediately at runtime, and returns the internal `Void` type. As with any void-returning call, `print(...)` may be used as a standalone expression statement but not as the right-hand side of an assignment.

2.4.5 Function Definitions and Calls

```
1 func_def -> 'def' IDENTIFIER '(' params? ')' block
2 params   -> IDENTIFIER (',' IDENTIFIER)* | epsilon
3 args     -> expr (',' expr)* | epsilon
```

The `?` suffix on `params?` in `func_def` (and on `args?` in the call form below) marks the construct as optional in EBNF form. The corresponding parse-tree renderer makes this explicit by emitting a single `epsilon` leaf as the only child of an empty `params` or `args` node, so an empty parameter or argument list is still labelled in the tree.

Function calls use value parameter passing. This means the argument values are evaluated before the call and copied into the function’s local environment.

The built-in `print()` function uses exactly the same surface syntax as other calls:

```
1 print(x);
2 print(3.14);
```

Return type uniformity. A function’s return type is inferred from its return statements. All return expressions within a function body must yield the same type; mixed return types are a compile-time error. A function with no return statement is assigned the special internal type `Void` and may only be invoked as a standalone statement, not used as an expression. Assigning the result of a `Void` function to a variable (e.g. `x = f();` where `f` returns nothing) is rejected by the type checker.

2.5 Operator Precedence and Associativity

Operator precedence is encoded structurally in the recursive-descent parser. Comparison is lowest, addition and subtraction are next, and multiplication and division have the highest precedence among binary operators. Arithmetic operators are left-associative.

Table 2.2. Operator Precedence (1 = lowest, 3 = highest) and Associativity

Precedence	Operators	Associativity
1 (lowest)	<code>==, !=</code>	non-associative (one comparison per expression)
2	<code>+, -, +., -.</code>	left-associative
3 (highest)	<code>*, /, *., /. </code>	left-associative

Table 2.3. Formal arithmetic and comparison operator signatures

Operator	Operand types	Result type
<code>+</code>	<code>Integer × Integer</code>	<code>Integer</code>
<code>-</code>	<code>Integer × Integer</code>	<code>Integer</code>
<code>*</code>	<code>Integer × Integer</code>	<code>Integer</code>
<code>/</code>	<code>Integer × Integer</code>	<code>Integer</code>
<code>+. </code>	<code>Float × Float</code>	<code>Float</code>
<code>-. </code>	<code>Float × Float</code>	<code>Float</code>
<code>*. </code>	<code>Float × Float</code>	<code>Float</code>
<code>/. </code>	<code>Float × Float</code>	<code>Float</code>
<code>==</code>	<code>Integer × Integer</code>	<code>Boolean</code>
<code>==</code>	<code>Float × Float</code>	<code>Boolean</code>
<code>!=</code>	<code>Integer × Integer</code>	<code>Boolean</code>
<code>!=</code>	<code>Float × Float</code>	<code>Boolean</code>

Any operand combination not listed in Table 2.3 is rejected during static type checking. Because each operator token maps to exactly one valid operand signature (or, for `==` and `!=`,

two same-type signatures), operator meaning can be determined without overload resolution or implicit type coercion.

2.6 Design Decisions: No Overloading

The assignment specification states: “No overloading with the usual precedence of multiplication and division over addition and subtraction. Hence no explicit declaration.” This section explains how the language design satisfies this requirement and why alternatives were rejected.

Definition of overloading. An operator is overloaded when the same syntactic symbol carries more than one type signature, requiring the compiler to inspect operand types to determine the operator’s meaning or result type. Under this definition, a language where $+$ means both integer addition $(Integer, Integer) \rightarrow Integer$ and floating-point addition $(Float, Float) \rightarrow Float$ — or worse, mixed-type addition $(Integer, Float) \rightarrow Float$ — overloads the $+$ symbol.

Our approach: distinct operator tokens. Following the Caml Light convention, this language assigns a unique token to each arithmetic meaning. Integer arithmetic uses $+$, $-$, $*$, $/$; floating-point arithmetic uses $+. , - . , * . , / .$. Each token has exactly one type signature. The type of any expression can be determined from the operator token alone, without inspecting operand types. This is the strongest possible interpretation of “no overloading.”

Rejected alternative: implicit numeric promotion. An alternative design would use a single set of operators and silently promote `Integer` operands to `Float` when mixed with a `Float` operand (e.g. $1 + 2.5 \rightarrow 3.5 : Float$). Proponents of this approach argue that promotion is not overloading because the operator retains a single “arithmetic addition” meaning. However, this argument is difficult to sustain formally: the operator $+$ would have three distinct input-type combinations — $(Integer, Integer)$, $(Integer, Float)$, and $(Float, Float)$ — with two different result types, a dispatch table that the compiler must resolve from operand types, which is the hallmark of overloading. The assignment specification also states “hence no explicit declaration,” implying that type inference should be unambiguous; implicit promotion introduces cases where the inferred result type depends on operand types rather than on the operator itself.

Connection to type inference. Because every operator token uniquely determines its operand and result types, the type inference algorithm described in Chapter 5 can determine the type of any expression without backtracking, unification, or constraint solving. This determinism directly satisfies the specification’s coupling of “no overloading” with “no explicit declaration.”

Table 2.4 contrasts the two designs.

Table 2.4. Comparison of operator design approaches

Expression	This language (separate tokens)	Promotion-based language
<code>3 + 4</code>	<code>7 : Integer (+ → Integer)</code>	<code>7 : Integer</code>
<code>3.0 +. 4.0</code>	<code>7.0 : Float (+. → Float)</code>	N/A (no <code>+. </code> operator)
<code>3 + 4.0</code>	<code>TypeError: + requires two Integer operands</code>	<code>7.0 : Float</code> (implicit promotion)
<code>10 / 3</code>	<code>3 : Integer</code> (floor division)	<code>3.333... : Float</code> (or <code>3 : Integer</code> , design-dependent)
Type signatures of <code>+</code>	One: $(Int, Int) \rightarrow Int$	Three: (Int, Int) , $(Int, Float)$, $(Float, Float)$
Overloaded?	No	Arguably yes

CHAPTER 3

LEXICAL ANALYSIS AND SYMBOL TABLE

This chapter covers the responsibilities related to lexical and early semantic foundations of the language. It includes defining token structures and categories (such as keywords, operators, identifiers, and literals), implementing the lexer to transform source characters into tokens, and establishing the symbol table to manage variable and function storage along with their associated types. Together, these components ensure that raw source input is systematically organised into meaningful structures that can be used reliably by later stages of the compiler.

3.1 Token representation

The token system, written in `components/tokens.py`, establishes the vocabulary and structure of all lexical elements in the language. It specifies both the types of tokens that can exist and the structure each token must follow. This serves as a canonical contract between the lexer and the parser, ensuring consistency across all stages of the compiler.

Table 3.1 summarises the main categories, example lexemes, and typical `TokenType` values.

Table 3.1. Token categories and representative `TokenType` values

Category	Example lexemes	Representative <code>TokenType</code>
Literals	42, 3.14, true, "hello"	INT_LITERAL, FLOAT_LITERAL, BOOL_LITERAL, STRING_LITERAL
Identifiers	x, counter, add	IDENTIFIER
Keywords	if, else, while, def, return	keyword-specific token variants
Integer operators	+, -, *, /	PLUS, MINUS, STAR, SLASH
Float operators	+, -, *, / .	PLUS_DOT, MINUS_DOT, STAR_DOT, SLASH_DOT
Other operators	=, ==, !=	ASSIGN, EQUAL_EQUAL, BANG_EQUAL
Delimiters	Parentheses, braces, comma, and semicolon (see lexer delimiter tokens)	delimiter token variants
End marker	end of source input	EOF

Token types (from class `TokenType`) are defined using an Enum with `auto()` to automatically assign unique values. This avoids reliance on raw strings and ensures safer comparisons during parsing. The token categories include literals, identifiers, keywords, operators, delimiters,

and a special end-of-file (EOF) token. The EOF token is appended after all input is processed, allowing the parser to detect the end of input explicitly rather than encountering an unexpected termination.

Each token is represented as an immutable dataclass (`frozen=True`), meaning its fields cannot be modified after creation. Each token contains four key pieces of information: its kind (`token_type`), its lexeme (the exact substring from the source code), and its source position (`line`, `column`). This structured representation allows later stages to process the program in terms of syntax and errors without handling raw text directly. Immutability improves reliability by preventing accidental modification during later processing.

```
1 @dataclass(frozen=True)
2 class Token:
3     token_type: TokenType
4     lexeme: str
5     line: int
6     column: int
```

3.2 Lexer implementation

The lexer, implemented in `components/lexica.py`, is a hand-written scanner that converts raw source text into a sequence of tokens. Unlike generated lexers (for example SLY lexer mode, or Lex and Flex), this implementation uses explicit control flow (`if` and `while`) to process input.

The lexer maintains internal state including the source string, current position, start of the current token, line number, and column index. The column index is tracked per line and resets upon encountering a newline, while a separate `token_column` records the starting position of each token for accurate error reporting.

The main function, `tokenize`, iterates through the source code, repeatedly invoking `_scan_token` to identify the next token. Tokens are appended to a list, and an EOF token is added at the end.

Whitespace is skipped, while invalid input results in a `LexerError`.

```
1 def tokenize(self) -> list[Token]:
2     tokens: list[Token] = []
3     while not self._is_at_end():
4         self.start = self.current
5         self.token_column = self.column
6         token = self._scan_token()
7         if token is not None:
8             tokens.append(token)
9     tokens.append(Token(TokenType.EOF, "", self.line, self.column))
```

```
10     return tokens
```

Token recognition follows a structured approach. Single-character tokens are mapped using a lookup table. Multi-character operators such as `==` and `!=` are handled using lookahead via `_match`. String literals are processed by `_string`, which reads characters until a closing quotation mark or the end of input is reached, with error handling for unterminated strings. Numeric literals are handled by `_number`, which distinguishes integers and floating-point values based on the presence of a decimal point followed by digits. Identifiers are processed greedily, consuming alphanumeric characters and underscores, and then checked against the `KEYWORDS` map to determine whether they are reserved words or user-defined names. Notably, `true` and `false` are also entries in this map, mapping to `TokenType.BOOL_LITERAL` rather than a keyword-specific type. This means boolean literals are recognised through the same keyword-fallback mechanism as reserved words, preventing them from being scanned as user-defined identifiers. In contrast, `print` is *not* reserved: it is scanned as an ordinary `IDENTIFIER` and later treated as a built-in callable by the type checker and interpreter.

```
1 KEYWORDS: dict[str, TokenType] = {
2     "if": TokenType.IF,
3     "else": TokenType.ELSE,
4     "while": TokenType.WHILE,
5     # ...
6     "true": TokenType.BOOL_LITERAL,
7     "false": TokenType.BOOL_LITERAL,
8 }

1 # Arithmetic operators with optional trailing '.' for float variant
2 _ARITH_DOT: dict[str, tuple[TokenType, TokenType]] = {
3     "+": (TokenType.PLUS, TokenType.PLUS_DOT),
4     "-": (TokenType.MINUS, TokenType.MINUS_DOT),
5     "*": (TokenType.STAR, TokenType.STAR_DOT),
6     "/": (TokenType.SLASH, TokenType.SLASH_DOT),
7 }
```

When one of these characters is encountered, the lexer checks the next character: if it is a `.` (and the character after that is not a digit), the dot-suffixed float variant is emitted; otherwise the plain integer operator is emitted. This lookahead prevents the `.` in a float literal such as `3.14` from being consumed as part of an operator.

```
1 SINGLE_CHAR_TOKENS: dict[str, TokenType] = {
2     "=": TokenType.ASSIGN,
3     "(": TokenType.LPAREN,
4     ")": TokenType.RPAREN,
5     # ...
6 }
```

Helper functions such as `_advance`, `_peek`, and `_peek_next` manage character consumption and lookahead, while `_is_at_end` determines when input has been fully processed. Error messages include precise line and column information, aiding debugging and diagnostics.

Table 3.2 lists the main functions and data structures in `components/lexica.py` and their roles.

Table 3.2. Key functions and data in `lexica.py`

Function / data	Responsibility	Output / effect
<code>tokenize()</code>	Iterates through source, scans tokens, appends EOF	ordered token list
<code>_scan_token()</code>	Dispatches recognition based on character and lookahead	next token or skip
<code>_number()</code>	Parses numeric sequences, distinguishes int vs float	numeric literal token
<code>_string()</code>	Parses quoted text, checks termination	string token or error
<code>_identifier()</code>	Parses identifiers, resolves keyword vs user-defined	keyword or identifier token
KEYWORDS map	Maps reserved words and Boolean literals to token types	token type selection
SINGLE_CHAR_TOKENS map	Maps single-character symbols	token type selection
<code>_advance()</code> , <code>_peek()</code> , <code>_match()</code>	Character consumption and lookahead	scanner state update

3.3 Symbol table and semantic structure

The symbol table, defined in `components/symbol_table.py`, operates at a higher level than the lexer. While the lexer processes raw text into tokens, the symbol table manages semantic information such as variable and function names, their types, and their scope.

The symbol table is implemented as a scoped structure, where each scope maintains its own dictionary of symbols and optionally references a parent scope. This enables proper handling of nested blocks and variable shadowing. Symbols are introduced through definition functions and retrieved through lookup operations, which search recursively through parent scopes if needed.

```

1 def lookup(self, name: str) -> Symbol:
2     symbol = self._symbols.get(name)
3     if symbol is not None:
4         return symbol
5     if self.parent is not None:

```

```

6         return self.parent.lookup(name)
7     raise SymbolTableError(f"Undefined symbol: {name}")

```

The type system is represented using a `LanguageType` enumeration, which includes fundamental types such as `Integer`, `Float`, `Boolean`, `String`, and `Void`. Symbols are modelled using an inheritance hierarchy: the base `Symbol` class stores a name and type, while subclasses such as `VariableSymbol` and `FunctionSymbol` include additional attributes. Variables track initialization state, while functions store parameter lists and return types. Table 3.3 summarises the main building blocks.

Table 3.3. Symbol table components and roles

Component	Stored information	Role in semantic analysis
Scoped symbol table	Symbols in current and parent scope	Supports nesting and shadowing via recursive lookup
<code>LanguageType</code> enum	<code>Integer</code> , <code>Float</code> , <code>Boolean</code> , <code>String</code> , <code>Void</code>	Defines the type system
<code>Symbol</code> (base)	Name and declared or inferred type	Base abstraction for entries
<code>VariableSymbol</code>	Variable metadata (including initialization state)	Tracks correct usage before assignment
<code>FunctionSymbol</code>	Parameters and return type	Validates calls and return consistency
<code>lookup(name)</code>	Recursive scope search	Resolves identifiers or reports undefined
<code>SymbolTableError</code>	Semantic error details	Reports duplicates, undefined symbols, invalid operations

Errors related to semantic violations are handled via `SymbolTableError`. These include duplicate declarations within the same scope, references to undefined symbols, and invalid operations such as attempting to mark a non-variable symbol as initialized.

3.3.1 Example symbol table hierarchy

Figure 3.1 illustrates a small two-scope symbol table after the type checker has processed a program containing two top-level variables and a function definition.

Global scope		
Name	Kind	Type
x	Variable	Integer
y	Variable	Float
addf	Function	Float $\times$ Float $\rightarrow$ Float
Function scope: addf		
Name	Kind	Type
a	Parameter	Float
b	Parameter	Float

Figure 3.1. Example symbol table hierarchy with one global scope and one function scope.

The global scope stores top-level variables and function definitions, while each function invocation creates a child scope containing parameter bindings and local variables. Lookup operations recursively search parent scopes when identifiers are not found in the current environment.

3.4 Role of the symbol table in the pipeline

The symbol table is constructed during the type-checking phase rather than during lexical analysis. This guideline is followed in the implementation. The type checker initialises a global symbol table and populates it while traversing the AST, including defining variables and functions, managing nested scopes, and validating type correctness.

```

1 def run_pipeline(source_code: str) -> PipelineResult:
2     tokens = Lexer(source_code).tokenize()
3     tree = Parser(tokens).parse()
4     parse_tree_text = ParseTreePrinter().print(tree)
5     ast_text = ASTPrinter().print(tree)
6     checked_scope = TypeChecker().check(tree)
7     outputs: list[str] = []
8     Interpreter(output_callback=outputs.append).execute(tree)
9     return PipelineResult(
10         tokens=tokens,
11         parse_tree_text=parse_tree_text,
12         ast_text=ast_text,
13         checked_scope=checked_scope,
14         outputs=outputs,
15     )

```

Once type checking is complete, the populated symbol table is returned as part of the pipeline result (`checked_scope`). The checker creates child scopes for blocks and functions during analysis, but the displayed `checked_scope` table represents the global scope entries returned

by the pipeline. It can then be displayed or used for further analysis.

This separation of concerns keeps lexical analysis focused on tokenization, while semantic meaning is enforced during type checking. For user-defined functions, definitions are first registered in the global scope with placeholder parameter and return types. These are inferred from the first valid call and refined through body checking; subsequent calls must match the inferred parameter types.

CHAPTER 4

RECURSIVE-DESCENT PARSING, PARSE TREE, AND ABSTRACT SYNTAX TREE

This chapter presents the implementation of the recursive-descent parser, the grammar-oriented parse-tree view exported for inspection, and the associated abstract syntax tree (AST) model used for downstream semantic analysis and execution. Starting from lexer-produced tokens, the parser constructs a hierarchical Program AST. A separate `ParseTreePrinter` then renders that parsed program in an ASCII branch-style rule-oriented tree form so that the submission can show explicit correspondence between grammar productions and accepted programs without changing the executable internal representation. Implemented syntax coverage includes arithmetic expressions, assignments, if-then-else, while, function definitions, function calls, return, and built-in `print(...)` calls. Equality operators (`==`, `!=`) are parsed as ordinary lowest-precedence expressions, so Boolean results can appear in conditions, assignments, function arguments, and `print(...)` expressions.

4.1 Abstract syntax tree representation

The AST is defined as a collection of Python dataclasses representing the structural elements of the programming language. It serves as an intermediate representation between parsing and later stages such as type checking and interpretation.

A key distinction exists between tokens and AST nodes. Tokens are flat and describe individual lexical elements such as operators or identifiers. In contrast, the AST is hierarchical. For example, while a token may represent the symbol `+`, a `BinaryOp` node represents an entire addition operation, including its left and right operands as subtrees.

All AST nodes inherit from a common base class, allowing them to store positional metadata (line, column). This ensures that errors detected in later stages can still reference precise locations in the original source code.

```
1 @dataclass
2 class ASTNode:
3     """Base class for every AST node. Carries source location for
4         diagnostics."""
5     line: int = field(default=0, kw_only=True)
```



```
5 column: int = field(default=0, kw_only=True)
```

4.1.1 Expressions

Expressions represent computations or values within the program. A `Literal` node stores values known at parse time, including integers, floating-point numbers, booleans, and strings. String literals are stored without surrounding quotation marks, as these are removed during parsing.

```
1 if tok.token_type == TokenType.STRING_LITERAL:
2     self._advance()
3     return Literal(value=tok.lexeme[1:-1], line=tok.line, column=tok.
        column)
```

An `Identifier` node represents the use of a variable name within an expression, indicating evaluation rather than declaration. A `BinaryOp` node models arithmetic and comparison operations such as addition, subtraction, multiplication, division, and equality checks, storing the operator along with references to left and right operands. A `FunctionCall` node represents function invocation, consisting of the function name and a list of argument expressions.

```
1 @dataclass
2 class BinaryOp(ASTNode):
3     """An arithmetic or comparison binary operation.
4
5     op is one of: '+' '-' '*' '/' '+.' '-.' '*.' '/.' '==' '!= '
6     Example:  a + b    x == 0    a +. b
7     """
8     op: str
9     left: ASTNode
10    right: ASTNode
```

4.1.2 Statements

Statements represent actions or control flow within the program. An `Assign` node models variable assignment, pairing a variable name with a value expression, while `Return` stores a single expression and represents function return. A `FunctionCall` node may also appear as a standalone statement, which is how built-in `print(...)` calls are represented after parsing. A `Block` node groups multiple statements within braces, representing a scoped execution sequence.

Control flow is handled via `If` and `While` nodes. The `If` node stores a condition, a then-block, and an optional else-block, while `While` stores a loop condition and body block. Function

definitions are represented by `FunctionDef` nodes, which include the function name, a list of parameter names, and a body block. Parameter types are intentionally not assigned at parse time, as type inference is handled later by the type checker. Finally, `Program` serves as the AST root, containing all top-level statements.

4.2 Parser implementation

The parser consumes the list of tokens generated by the lexer and produces a single `Program` AST node. It is implemented as a hand-written recursive-descent parser based on the project grammar. This avoids parser generators and instead relies on explicit functions for each grammar rule.

The parser maintains internal state consisting of the token list and a position index (`self._pos`) indicating the current token. Tokens are accessed via helper methods such as `_peek` and `_advance`. Parsing errors are reported using a custom `ParseError` exception with line and column information consistent with lexer diagnostics.

```
1 def parse(self) -> Program:
2     """Parse the full token stream and return the root Program node."""
3     line, col = self._peek().line, self._peek().column
4     statements: list[ASTNode] = []
5     while not self._is_at_end():
6         statements.append(self._statement())
7     return Program(statements=statements, line=line, column=col)
```

In addition to the main `parse()` entry point, the parser's statement dispatcher (`_statement`) implements a deterministic decision order over token patterns: function definition, conditional, loop, return, assignment, then expression statement fallback. This design is important because it prevents ambiguity between syntactically similar forms. In particular, assignments are recognized using one-token lookahead (`IDENTIFIER` followed by `=`), while identifier-led expressions that are not assignments are parsed as expression statements and must still end with a semicolon. This gives the parser full coverage of both declarative-style and expression-driven top-level statements without requiring a parser generator.

4.2.1 Rationale for recursive-descent parsing

Recursive-descent parsing was selected because the project grammar is sufficiently structured to permit predictive parsing with explicit precedence layering. This approach avoids parser generators and keeps parsing logic directly aligned with grammar productions, making the

implementation easier to debug, extend, and explain.

Operator precedence is encoded structurally through layered parsing methods (`_expr`, `_additive`, `_term`, `_factor`), eliminating the need for runtime precedence tables. The hand-written approach also provides explicit control over lookahead and ambiguity resolution, particularly for distinguishing assignments from identifier-led expression statements.

4.3 Parse-tree view

Although the parser’s internal product is an AST, the project now also exposes a separate parse-tree view for reporting and demonstration. This view is generated by `components/parse_tree_printer.py` from the parsed program and uses grammar-oriented layers such as `program`, `statement`, `assignment`, `expr`, `comparison`, `additive`, `term`, `factor`, `function_call`, and `args`. The `comparison` layer wraps the operands of `==` and `!=` when they appear at the top of an expression. Terminal-like leaves are rendered as `IDENTIFIER(x)`, `INT_LITERAL(5)`, `'(', ')', ';'`, and `EOF`, with ASCII branch markers indicating parent-child structure.

This design keeps the semantic implementation simple: the type checker and interpreter still consume the AST, while the parse-tree renderer exists purely to expose the grammar structure required for inspection.

Table 4.1. Parse tree versus AST in the implemented system

Artifact	Purpose	Used by
Parse tree	Grammar-oriented inspection view	CLI, UI, report
AST	Executable semantic tree	Type checker, interpreter

For presentation purposes, Figure 4.1 gives a small diagram for the source fragment `a = 3;`. This figure is included only as an explanatory visual in the report; the actual system output uses the more scalable ASCII branch representation shown in the CLI and UI.

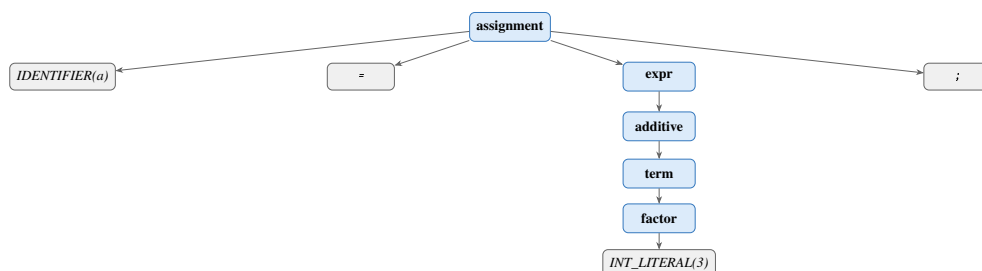


Figure 4.1. Parse-tree diagram for `a = 3;`.

4.3.1 Statement parsing

The main parsing loop repeatedly processes statements until EOF, with the statement type determined via dispatch on the current token. Function definitions, conditionals, loops, and returns each have dedicated parsing methods. Assignments are detected using one-token lookahead (IDENTIFIER followed by ASSIGN), preventing misclassification of function calls as assignments. If no specific statement pattern matches, the parser falls back to an expression statement and enforces a terminating semicolon.

```
1 # assignment: IDENTIFIER "=" expr ";"
2 if (
3     tok.token_type == TokenType.IDENTIFIER
4     and self._peek_next().token_type == TokenType.ASSIGN
5 ):
6     return self._assignment()
7 # fallback: bare expression statement expr ";"
8 node = self._expr()
9 self._expect(TokenType.SEMICOLON, "Expected ';' after expression")
```

4.3.2 Expression parsing and precedence

Expressions are parsed in layered precedence levels: `_expr` handles comparison operators first (`==`, `!=`) after parsing an additive left-hand side, then `_additive` handles `+`, `-`, `+. .`, and `-.`; `_term` handles `*`, `/`, `*. .`, and `/. .`; and `_factor` handles literals, identifiers and function calls, and parenthesized expressions. Function calls are recognised when an identifier is followed by `(`, with argument lists parsed as comma-separated expressions. Parenthesized expressions are supported to override precedence. Unary negation (`-x`, `-5`) is also handled at the `_factor` level: a leading `-` token causes the parser to recursively parse the operand and return `BinaryOp(0 - operand)`, so the existing type rules for subtraction cover negation without any additional inference code. Float unary negation is handled similarly for `-.`, producing `BinaryOp(0.0 - . operand)`.

```
1 def _expr(self) -> ASTNode:      # ==, !=
2     ...
3 def _additive(self) -> ASTNode:  # +, -, +. ., -.
4     ...
5 def _term(self) -> ASTNode:     # *, /, *. ., /.
6     ...
7 def _factor(self) -> ASTNode:   # unary -, literals, identifiers/calls,
8     grouped expr
9     ...
```

Figure 4.2 shows the AST produced for the statement `z = x + y * 2;`. Because `_term` han-

dles `*` before `_additive` handles `+`, the multiplication sub-tree is nested one level deeper, encoding the higher precedence of `*` over `+` without any explicit priority table at runtime.

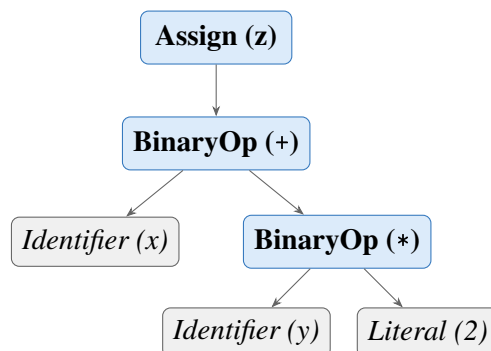


Figure 4.2. AST for `z = x + y * 2;`. Multiplication is a deeper sub-tree than addition, encoding higher precedence through tree structure rather than a runtime priority table.

This structural encoding of precedence eliminates the need for runtime operator-priority resolution and keeps semantic analysis independent of parsing strategy.

Expression parsing is intentionally split into layered methods to enforce precedence and associativity in a transparent way. Parenthesized expressions are parsed explicitly and re-enter `_expr`, allowing precedence override where required by source syntax. Comparison operators (`==`, `!=`) are parsed at the top of the `_expr` method as the lowest-precedence binary operators. Because comparisons are part of the general expression grammar, Boolean results can appear in any expression context — as conditions, assignments, arguments, or print expressions. Semantic enforcement that a condition is truly Boolean is deferred to the type checker.

Figure 4.3 shows the AST for an `if` statement with a comparison condition, illustrating how the parser structures control flow with a Boolean sub-tree.

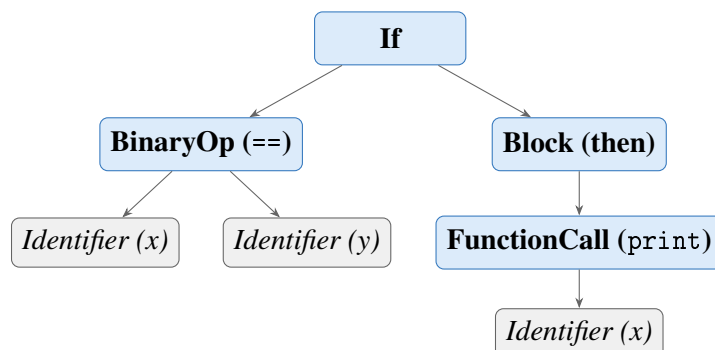


Figure 4.3. AST for `if (x == y) { print(x); }`. The comparison produces a **BinaryOp** node whose inferred type is **Boolean**, checked by the type checker before execution.

4.3.3 *Helper functions and lookahead*

As described above, the parser uses `_check`, `_expect`, `_peek`, `_peek_next`, `_advance`, and `_is_at_end` for token navigation and validation. Lookahead is central for ambiguity resolution: assignment versus function call is resolved by inspecting the next token, while comparison operators are recognised in `_expr` after parsing the left additive expression. Parser errors follow a consistent diagnostic format: `[line X, col Y] ParseError: <message>`.

The parser also includes dedicated list parsers for formal parameters (`_params`) and call arguments (`_args`), both implemented as comma-separated sequences with optional emptiness. This ensures consistent handling of signatures such as `def f()` and `def f(a, b)` as well as invocations like `f()` and `f(x, y+1)`.

4.4 AST printing

For debugging and reporting, an AST printer converts the tree into a readable indented string, integrated into the pipeline output. The printer uses visitor-style dynamic dispatch, mapping node class names to methods such as `_visit_If` and `_visit_BinaryOp`, with output built as multi-line text where indentation reflects tree depth. Node formatting emphasises structure: `Program` and `Block` include statement counts, `BinaryOp` explicitly shows left and right subtrees, and `FunctionCall` prints each argument by index. `Literal` includes both value and runtime Python type names for clarity.

4.5 Parser and type checker boundary

A deliberate architectural boundary is maintained between syntax construction and semantic validation. The parser is responsible for producing a structurally valid AST with accurate source locations. The parse-tree renderer exposes the same parsed program in grammar-facing form, but it is not used for execution. Type correctness and semantic constraints are checked later by the type checker. For example, `_expr` may syntactically produce a node whose eventual type is not `Boolean`, but rejection of non-`Boolean` control-flow conditions is handled during semantic analysis. This separation keeps parser logic focused on grammar conformance and allows a clear responsibility split across team member contributions.

CHAPTER 5

TYPE INFERENCE ALGORITHM

5.1 Overview

Type inference in this project means that the programmer does not write explicit type declarations for variables or functions. Instead, the compiler infers types from assignments, literals, operations, function calls, and return statements. This keeps the language simple while still preserving static type safety.

The type checker runs after parsing. It receives the AST and validates the program before execution. Any inconsistent use of types is reported as a type error, and execution is stopped.

The interpreter stage does not repeat this validation: it walks the same AST with ordinary Python representations of values. Static type information remains in the symbol table produced by the checker; run-time values are not separately labelled with `LanguageType`, except that `print()` formats output by inspecting the value's Python type.

5.2 Type Environment

The type environment is represented by the symbol table. Conceptually, it is a mapping

$$\Gamma : \text{Identifier} \rightarrow \text{Type}$$

The environment grows as the checker visits the AST. On first assignment, a variable name is inserted with its inferred type. When a new block or a function call is entered, a child environment is created so that nested scopes can be checked independently while still accessing outer bindings when needed.

5.3 Type Inference Procedure

The type checker performs type inference by traversing the abstract syntax tree before interpretation. The algorithm is syntax-directed: each AST node is visited, the types of its child nodes are inferred first, and the current node is checked according to the language typing rules.

Algorithm TYPECHECK(*node*, Γ)

Input: *node* = current AST node; Γ = current type environment / symbol table

Output: inferred type of *node*, or `TypeError`

1. **If** *node* is a **Literal**: return the corresponding primitive type: `Integer`, `Float`, `Boolean`, or `String`.
2. **If** *node* is an **Identifier**: look up the identifier in Γ . If it is not found, report an undefined variable error. Otherwise, return its stored type.
3. **If** *node* is a **BinaryOp**:
 - Infer the type of the left operand and the right operand.
 - If the operator is `+`, `-`, `*`, or `/`: require both operands to be `Integer`; return `Integer`.
 - If the operator is `+`, `-`, `*`, or `/`: require both operands to be `Float`; return `Float`.
 - If the operator is `==` or `!=`: require both operands to be the same arithmetic type; return `Boolean`.
 - Otherwise, report a `TypeError`.
4. **If** *node* is an **Assignment**:
 - Infer the type of the right-hand-side expression.
 - If the variable is not yet in Γ : insert it with the inferred type.
 - Else: verify the new type matches the existing type; if not, report a `TypeError`.
5. **If** *node* is an **If** statement:
 - Infer the condition type; require it to be `Boolean`.
 - Type-check the then-block in a child environment.
 - If an else-block exists, type-check it in a child environment.
6. **If** *node* is a **While** statement:
 - Infer the condition type; require it to be `Boolean`.
 - Type-check the loop body in a child environment.
7. **If** *node* is a **FunctionDef**:
 - Register the function name in Γ .
 - Parameter types are initially unknown.
 - Return type is inferred from return statements in the body.

8. If *node* is a **FunctionCall**:

- If the function is the built-in `print`: require exactly one argument, infer that argument type, and return `Void`.
- Otherwise, infer all argument types, validate them against the function signature, type-check the callee body if needed, and return the inferred function return type.

5.3.1 Example type inference trace

Table 5.1 traces the algorithm step by step for the program fragment

```
1 x = 3;  
2 y = 4;  
3 z = x + y;
```

showing how the type environment Γ grows as each sub-expression is inferred.

Table 5.1. Step-by-step type inference trace for `x = 3; y = 4; z = x + y;`

Step	Expression	Inferred type	Environment Γ
1	3	Integer	$\emptyset$
2	x = 3	Integer	$\{x: \text{Integer}\}$
3	4	Integer	$\{x: \text{Integer}\}$
4	y = 4	Integer	$\{x: \text{Integer}, y: \text{Integer}\}$
5	x + y	Integer	unchanged
6	z = x + y	Integer	$\{x: \text{Integer}, y: \text{Integer}, z: \text{Integer}\}$

5.4 Inference Rules

5.4.1 Literals

Literal nodes have direct types:

$\Gamma \vdash 42 : \text{Integer}$

$\Gamma \vdash 3.14 : \text{Float}$

$\Gamma \vdash \text{true} : \text{Boolean}$

$\Gamma \vdash \text{"hi"} : \text{String}$

5.4.2 No Operator Overloading

The language does not support operator overloading. Following the Caml Light convention, integer and float arithmetic use entirely separate operator tokens, so every operator has a single, fixed type signature:

- $+$, $-$, $*$, $/$ — require two `Integer` operands; result is `Integer`.
- $+. , -. , *. , /.$ — require two `Float` operands; result is `Float`.
- $+$ is not overloaded for string concatenation; applying it to a `String` is a type error.
- $==$ and $!=$ are not overloaded for `Boolean` or `String` equality.
- No implicit numeric promotion: `Integer + Float` is a type error.

Because every operator has a unique, unambiguous type derivable from its token alone, the compiler can always infer expression types without requiring explicit type declarations from the programmer. This design ensures that operator meaning is determined uniquely from the operator token itself, eliminating semantic ambiguity during type inference and simplifying static analysis.

5.4.3 Arithmetic Expressions

The integer operators $+$, $-$, $*$, and $/$ require both operands to be `Integer` and always return `Integer` (including division, which uses integer floor division). The float operators $+. , -. , *. , /.$ require both operands to be `Float` and always return `Float`. Mixing an `Integer` and a `Float` with any operator is a type error.

Table 5.2. Arithmetic operator type rules

Left operand	Operator	Right operand	Result type
Integer	$+, -, *, /$	Integer	Integer
Float	$+. , -. , *. , /. $	Float	Float
Integer	any	Float	TypeError
Float	any	Integer	TypeError
Any	any	String or Boolean	TypeError

The following formal typing judgements capture these rules precisely. For integer operators ($\text{op} \in \{+, -, *, /\}$):

$$\frac{\Gamma \vdash e_1 : \text{Integer} \quad \Gamma \vdash e_2 : \text{Integer}}{\Gamma \vdash e_1 \text{ op } e_2 : \text{Integer}}$$

For float operators ($\text{op} \in \{+., -., *, /. \}$):

$$\frac{\Gamma \vdash e_1 : \text{Float} \quad \Gamma \vdash e_2 : \text{Float}}{\Gamma \vdash e_1 \text{ op } e_2 : \text{Float}}$$

For mixed-type operands with any operator (and symmetrically for *Float/Integer*):

$$\frac{\Gamma \vdash e_1 : \text{Integer} \quad \Gamma \vdash e_2 : \text{Float}}{\text{TypeError}}$$

Note that each rule is keyed on the operator token, not on the operand types. This is only possible because integer and float arithmetic use distinct tokens — the property that eliminates operator overloading.

Unary negation

Unary integer negation ($-x$) is desugared by the parser to $0 - x$. Unary float negation ($-.x$) is desugared to $0.0 -. x$. The type checker therefore applies the standard binary rules in both cases.

5.4.4 Boolean Expressions

The comparison operators `==` and `!=` always produce `Boolean`. Both operators require two operands of the *same* arithmetic type — either both `Integer` or both `Float`; mixed-type comparisons such as `Integer == Float` are rejected. Comparisons involving `Boolean` or `String` operands are also rejected by the type checker. Although `Boolean` and `String` values exist in the language, equality and inequality are deliberately restricted to arithmetic expressions because the project specification defines basic `Boolean` expressions as equality and inequality between two arithmetic expressions.

Furthermore, the same-type requirement for comparisons — both operands must be either both `Integer` or both `Float` — is a direct consequence of the no-overloading design. If `==` accepted mixed `Integer` and `Float` operands, the compiler would need to determine whether to compare by promotion (implicitly converting `Integer` to `Float`) or by value identity, introducing the same ambiguity that the separate-operator design eliminates for arithmetic. Rejecting mixed-type comparisons ensures that `==` and `!=` each have exactly two valid type signatures — $(\text{Integer}, \text{Integer}) \rightarrow \text{Boolean}$ and $(\text{Float}, \text{Float}) \rightarrow \text{Boolean}$ — rather than a third mixed-type signature.

5.4.5 Assignment Statement

Assignments define variable types on first use. If a variable does not yet exist in the type environment, the checker infers the type of the right-hand side and inserts the variable into the environment. If the variable already exists, the new value must have the same type.

If $x \notin \Gamma$ and $\Gamma \vdash e : T$, then $\Gamma, x : T \vdash x = e$

If $x : T \in \Gamma$ and $\Gamma \vdash e : T$, then $\Gamma \vdash x = e$

5.4.6 If-Then-Else

The condition of an if statement must be Boolean. The then-block and else-block are checked in child scopes so that local operations can be validated cleanly.

$$\Gamma \vdash c : \text{Boolean} \quad \Gamma' \vdash B_{\text{then}} \quad \Gamma'' \vdash B_{\text{else}} \implies \Gamma \vdash \text{if}(c) B_{\text{then}} \text{ else } B_{\text{else}}$$

5.4.7 While-Loop

The condition of a while loop must also be Boolean. The loop body is checked in a child scope and must not violate the types of variables already introduced in the surrounding environment.

$$\Gamma \vdash c : \text{Boolean} \quad \Gamma' \vdash B \implies \Gamma \vdash \text{while}(c) B$$

5.4.8 Function Definition and Call

Function parameter types are inferred from the first call to the function. After that first call, subsequent calls must use arguments of the same types. The return type is inferred from the return statements in the function body. If the function returns inconsistent types, the checker reports an error.

For functions that are defined but never called, the type checker performs a post-pass after all top-level statements have been processed. It infers parameter types by scanning the body for arithmetic and comparison usage, then checks the full body with those inferred types. This ensures that type errors inside never-called function bodies are still detected before execution.

Recursive calls are intentionally outside the scope of this implementation; supporting them would require additional fixed-point inference or explicit recursion handling that is orthogonal to the core type-inference algorithm.

Value parameter passing example

The language uses value parameter passing: each argument is evaluated in the caller's environment and a copy of its value is bound to the corresponding parameter in the callee's environment. Figure 5.1 shows the runtime environment before, during, and after a call to a function that mutates its parameter.

Source program:

```
1 x = 5;  
2  
3 def f(a) {  
4     a = a + 1;  
5 }  
6  
7 f(x);
```

```
Before the call  
Global scope:    x → 5  
  
During the call  
Global scope:   x → 5  
Function scope (f):  a → 5, then a → 6  
  
After the call  
Global scope:    x → 5
```

Figure 5.1. Value parameter passing: the parameter *a* receives a copy of *x*, so mutating *a* inside *f* does not affect *x* in the caller.

Because the parameter receives a copy of the argument value rather than a shared reference, modifications to *a* inside the function do not affect *x* in the caller environment. This demonstrates standard value parameter passing semantics and avoids aliasing effects associated with reference-based mechanisms.

5.5 Type Error Reporting

Type errors are reported with source positions so that the user can locate the problem quickly. The format used by the system is:

[line X, col Y] TypeError: message

Examples include assigning a `String` to an `Integer` variable, using a non-Boolean condition in an `if` statement, or supplying the wrong argument type to a function.

Table 5.3. Representative type errors detected by the static type checker

Invalid program	Reason	Detection stage
<code>x = 5; x = "hello";</code>	Inconsistent variable type	Type checker
<code>3 + 4.0</code>	Mixed arithmetic types	Type checker
<code>if (3) { ... }</code>	Condition not Boolean	Type checker
<code>print()</code>	Wrong argument count	Type checker
<code>x = print(5);</code>	Void assigned to variable	Type checker
<code>true == false</code>	Boolean comparison unsupported	Type checker

5.6 Example Type Inference Traces

Consider the following program fragment:

```

1 x = 3;
2 y = 4;
3 z = x + y;
4 print(z);

```

The checker infers:

- `x`: Integer (from literal 3)
- `y`: Integer (from literal 4)
- `x + y`: Integer (Integer + Integer $\rightarrow$ Integer)
- `z`: Integer

For a float example using dot-suffixed operators:

```

1 def addf(a, b) {
2     return a +. b;
3 }
4
5 result = addf(2.0, 3.5);

```

The first function call infers `a`: Float and `b`: Float. The operator `+.` requires both operands to be Float, so the function return type is inferred as Float. The variable `result` is also inferred as Float.

CHAPTER 6

SYSTEM ARCHITECTURE AND IMPLEMENTATION

This chapter describes how the implementation is organised: the end-to-end processing flow, repository layout, technology choices, and the main components that realise static analysis and execution (type checker, interpreter, and user interfaces). Detailed treatment of lexical analysis, the symbol table, and parsing appears in Chapters 3 and 4; Chapter 5 presents the type-inference rules that the type checker implements.

6.1 Compiler pipeline

The system follows a pipeline with five stages, each exposing a labelled output section through the shared pipeline. Lexical and syntactic processing are kept separate from semantic analysis and execution, which simplifies testing and localises errors to the appropriate stage.

The overall flow of data through the system is shown in Figure 6.1.

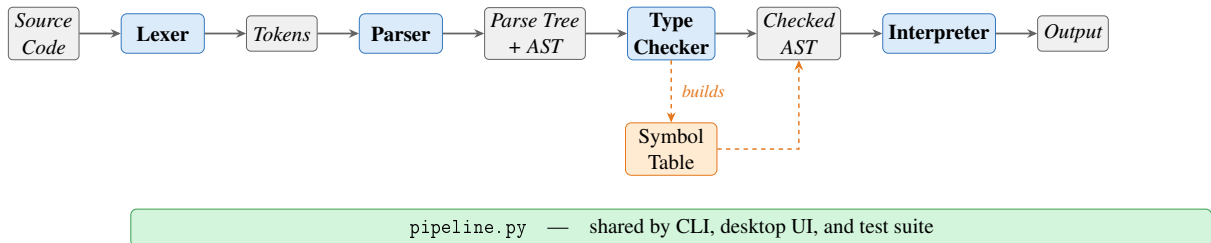


Figure 6.1. Front-end and interpreter pipeline overview.

Source characters are scanned by the lexer into a token stream. The parser consumes tokens and builds an AST that reflects program structure (expressions, statements, and control flow). The shared pipeline also renders a separate parse-tree view from the parsed program so that grammar structure can be inspected explicitly in the CLI, UI, and report. The type checker then walks the AST, populates and uses the symbol table, infers types, and validates scope and type rules. Only programs that pass the type checker proceed to interpretation. The interpreter evaluates the checked tree using ordinary Python values; it does not re-run the type rules, since correctness at runtime is guaranteed by the prior static pass. All stages are orchestrated through `pipeline.py`, which is shared by the command-line runner, the desktop UI, and the test suite.

6.1.1 End-to-end pipeline walkthrough

The following example traces the complete execution of a five-statement program through the exported pipeline views and execution stages, illustrating the correspondence between source text, tokens, parse tree, abstract syntax tree, inferred types, and runtime output.

Source program:

```
1 a = 3;
2 b = 4;
3 print(a + b);
4 c = 1.5;
5 print(c +. c);
```

Stage 1 — Lexer output (token stream):

1	IDENTIFIER	'a'	line=1	col=1
2	ASSIGN	'='	line=1	col=3
3	INT_LITERAL	'3'	line=1	col=5
4	SEMICOLON	';'	line=1	col=6
5	IDENTIFIER	'b'	line=2	col=1
6	ASSIGN	'='	line=2	col=3
7	INT_LITERAL	'4'	line=2	col=5
8	SEMICOLON	';'	line=2	col=6
9	IDENTIFIER	'print'	line=3	col=1
10	LPAREN	'('	line=3	col=6
11	IDENTIFIER	'a'	line=3	col=7
12	PLUS	'+'	line=3	col=9
13	IDENTIFIER	'b'	line=3	col=11
14	RPAREN	')'	line=3	col=12
15	SEMICOLON	';'	line=3	col=13
16	IDENTIFIER	'c'	line=4	col=1
17	ASSIGN	'='	line=4	col=3
18	FLOAT_LITERAL	'1.5'	line=4	col=5
19	SEMICOLON	';'	line=4	col=8
20	IDENTIFIER	'print'	line=5	col=1
21	LPAREN	'('	line=5	col=6
22	IDENTIFIER	'c'	line=5	col=7
23	PLUS_DOT	'+. '	line=5	col=9
24	IDENTIFIER	'c'	line=5	col=12
25	RPAREN	')'	line=5	col=13
26	SEMICOLON	';'	line=5	col=14
27	EOF	''	line=5	col=15

Stage 2 — Parser output (ASCII branch parse tree):

```
1 program
2 |-- statement
3 |   '-- assignment
4 |       |-- IDENTIFIER(a)
5 |       |-- '='
6 |       |-- expr
```



```

7 |         |         '-- additive
8 |         |         '-- term
9 |         |         '-- factor
10 |        |         '-- INT_LITERAL(3)
11 |        '-- ';'
12 |-- statement
13 |    '-- assignment
14 |        |-- IDENTIFIER(b)
15 |        |-- '='
16 |        |-- expr
17 |            |-- additive
18 |                |-- term
19 |                    |-- factor
20 |                        '-- INT_LITERAL(4)
21 |        '-- ';'
22 |-- statement
23 |    '-- expr_stmt
24 |        |-- expr
25 |            |-- additive
26 |                |-- term
27 |                    |-- factor
28 |                        '-- function_call
29 |                            |-- IDENTIFIER(print)
30 |                            |-- '('
31 |                            |-- args
32 |                                '-- expr
33 |                                    '-- additive
34 |                                        |-- additive
35 |                                            '-- term
36 |                                                '-- factor
37 |                                                    '-- IDENTIFIER(a)
38 |                                                        |-- '+'
39 |                                                            '-- term
40 |                                                                '-- factor
41 |                                                                    '-- IDENTIFIER(b)
42 |                            '-- ')
43 |        '-- ';'
44 |-- statement
45 |    '-- assignment
46 |        |-- IDENTIFIER(c)
47 |        |-- '='
48 |        |-- expr
49 |            |-- additive
50 |                |-- term
51 |                    |-- factor
52 |                        '-- FLOAT_LITERAL(1.5)
53 |        '-- ';'
54 |-- statement
55 |    '-- expr_stmt
56 |        |-- expr
57 |            |-- additive
58 |                |-- term
59 |                    |-- factor

```

```
60 |         '-- function_call  
61 |         |-- IDENTIFIER(print)  
62 |         |-- '('  
63 |         |-- args  
64 |         |--     expr  
65 |             |-- additive  
66 |                 |-- additive  
67 |                     |-- term  
68 |                         |-- factor  
69 |                             |-- IDENTIFIER(c)  
70 |                                 |-- '+.'  
71 |                                     |-- term  
72 |                                         |-- factor  
73 |                                             |-- IDENTIFIER(c)  
74 |                                                 '-- ')'  
75 |     '-- ';' ,  
76 | -- EOF
```

Stage 3 — Parser output (abstract syntax tree):

```

1 Program [5 statement(s)]
2     Assign 'a' =
3         Literal 3 (int)
4     Assign 'b' =
5         Literal 4 (int)
6     FunctionCall 'print' [1 arg(s)]
7         arg[0]:
8             BinaryOp '+'
9             left:
10                 Identifier 'a'
11             right:
12                 Identifier 'b'
13     Assign 'c' =
14         Literal 1.5 (float)
15     FunctionCall 'print' [1 arg(s)]
16         arg[0]:
17             BinaryOp '+.'
18             left:
19                 Identifier 'c'
20             right:
21                 Identifier 'c'

```

Stage 4 — Type checker output (inferred type environment):

Name	Inferred type
a	Integer
b	Integer
c	Float

The symbol table records only named symbols (a, b, and c). The type checker additionally

infers that the sub-expression `a + b` has type `Integer` (`Integer + Integer → Integer`) and that `c +. c` has type `Float` (`Float +. Float → Float`) during AST traversal, but sub-expression types are not stored in the symbol table.

Stage 5 — Interpreter output:

```
1 7 : Integer
2 3.0 : Float
```

The example demonstrates the full data flow: source characters are tokenised by the lexer, structured by the parser into both an inspectable parse tree and an executable AST, validated and annotated by the type checker, and finally evaluated by the interpreter to produce typed output. It also shows that the ASCII branch parse tree preserves the concrete operator spellings `'+'` and `'+. '`, making the distinction between integer and float arithmetic visible in the exported parser view.

Float operator walkthrough

To illustrate how the dot-suffixed float operators flow through each pipeline stage, consider a second example:

Source program:

```
1 x = 2.0;
2 y = 3.5;
3 print(x +. y);
```

Stage 1 — Lexer output (token stream):

1	IDENTIFIER	'x'	line=1 col=1
2	ASSIGN	'='	line=1 col=3
3	FLOAT_LITERAL	'2.0'	line=1 col=5
4	SEMICOLON	';'	line=1 col=8
5	IDENTIFIER	'y'	line=2 col=1
6	ASSIGN	'='	line=2 col=3
7	FLOAT_LITERAL	'3.5'	line=2 col=5
8	SEMICOLON	';'	line=2 col=8
9	IDENTIFIER	'print'	line=3 col=1
10	LPAREN	'('	line=3 col=6
11	IDENTIFIER	'x'	line=3 col=7
12	PLUS_DOT	'+. '	line=3 col=9
13	IDENTIFIER	'y'	line=3 col=12
14	RPAREN	')'	line=3 col=13
15	SEMICOLON	';'	line=3 col=14
16	EOF	''	line=3 col=15

Note that the lexer emits `PLUS_DOT` (not `PLUS`) for `+.` , distinguishing float addition from integer

addition at the token level. Note also that `print` is tokenised as an ordinary `IDENTIFIER` — it is not a reserved keyword in this language.

Stage 3 — Parser output (abstract syntax tree):

```
1 Program [3 statement(s)]
2   Assign 'x' =
3     Literal 2.0 (float)
4   Assign 'y' =
5     Literal 3.5 (float)
6   FunctionCall 'print' [1 arg(s)]
7     arg[0]:
8       BinaryOp '+.'
9         left:
10           Identifier 'x'
11         right:
12           Identifier 'y'
```

The AST records the operator as `+.` rather than `+`, preserving the type distinction through to semantic analysis.

Stage 4 — Type checker output (inferred type environment):

Name	Inferred type
x	Float
y	Float

The type checker verifies that both operands of `+.` are `Float`. Had either operand been `Integer`, a `TypeError` would be reported before execution.

Stage 5 — Interpreter output:

```
1 5.5 : Float
```

This second walkthrough demonstrates that the dot-suffixed operators are visible at every pipeline stage — from the lexer token `PLUS_DOT` to the AST node `BinaryOp '+. '` to the final typed output — ensuring that integer and float arithmetic are kept strictly separate throughout the system.

6.1.2 Entry points and shared pipeline

All stages are orchestrated by `components/pipeline.py`, which is shared by the command-line runner, the desktop UI, and the test suite. From the project root, the CLI and desktop UI can be launched as follows:

```

1 # macOS / Linux
2 python3 main.py          # run CLI (built-in sample or script file)
3 python3 ui.py            # open desktop workbench
4
5 # Windows
6 python main.py
7 python ui.py

```

The CLI prints five labelled stages in order: tokens, parse tree, AST text, type table, and execution output. The desktop workbench uses the same pipeline internally, presenting each stage in a separate tab for inspection.

6.2 Project structure

The main source files and their responsibilities are summarised in Table 6.1.

Table 6.1. Main source files

File	Responsibility
main.py	Command-line runner for sample code or a script file
ui.py	Desktop UI for editing and running a script
components/tokens.py	Token classes and token categories
components/lexica.py	Lexical analysis
components/ast_nodes.py	AST node definitions
components/parser.py	Recursive-descent parser
components/parse_tree_printer.py	Parse-tree renderer for inspection
components/ast_printer.py	AST renderer for debugging and reporting
components/symbol_table.py	Scoped symbol table and semantic types
components/type_checker.py	Static type checker and inference
components/interpreter.py	Tree-walking interpreter
components/pipeline.py	Shared pipeline for CLI, UI, and tests

6.3 Technology stack

The project uses Python and standard-library tools only (Table 6.2).

Table 6.2. Technology stack

Tool	Purpose
Python 3.10+	Core implementation language
dataclasses	AST and runtime helper structures
tkinter	Desktop graphical interface
unittest	Automated testing

6.4 Type checker

The `TypeChecker` class implements the inference and checking rules described in Chapter 5. It traverses the AST before execution, uses the symbol table to record variable and function information, infers types on first assignment, validates arithmetic and comparison expressions, enforces Boolean conditions for control flow, infers function parameter types from the first call (or from body usage for uncalled functions), and infers function return types from `return` statements. Programs that fail these rules are rejected before interpretation.

6.4.1 Scope of the implemented type system

In scope terms, this component turns a syntactically valid AST into a semantically valid program:

- static typing rules for arithmetic, comparison, assignment, control flow, and functions,
- type checking for expressions and statements,
- variable type inference from first assignment,
- function parameter inference from the first call (or from body usage for uncalled functions),
- function return type inference from `return` statements,
- source-positioned type errors when a program violates a typing rule.

6.5 Interpreter

The interpreter is a tree-walking evaluator over the checked AST. It maintains nested runtime environments for variables and function calls. Assignments write through the current or nearest enclosing scope; `if` selects a branch; `while` repeats until its condition becomes false; function calls bind arguments by value and return through an internal return mechanism.

The built-in `print()` function outputs both value and runtime type. All four types are supported:

```
1 print(42);  
2 print(3.14);  
3 print(true);  
4 print("plc");
```

These produce the output lines 42 : Integer, 3.14 : Float, true : Boolean, and "plc" : String respectively.

6.5.1 Scope of the implemented runtime

The runtime stage provides observable behaviour and integrates with the shared pipeline:

- assignment execution,
- if execution,
- while execution,
- function execution with value parameter passing,
- `print()` execution with immediate output,
- execution behind the same `run_pipeline` path used by the CLI, UI, and tests.

6.6 Graphical user interface

The desktop workbench (`ui.py`) is implemented with Python's `tkinter` standard library and requires no additional dependencies. The window is titled *125970-126690-126687-Ass2-PLC* and opens at 1200×760 pixels in a horizontal split layout.

Toolbar. A top toolbar provides three buttons — *Open Script*, *Run*, and *Clear Output* — and a right-aligned status label that reports *Ready* on launch, *Loaded <filename>* after a script is opened, *Ran <filename>* after a successful run on a loaded script file, *Ran editor contents* when running the editor buffer directly without having opened a file, *Run failed* when a pipeline stage raises an error, and *Output cleared* after the *Clear Output* button is pressed.

Left panel (source editor). A monospaced Text widget with both horizontal and vertical scrollbars allows the user to enter or paste arbitrary source code. The *Open Script* button populates the editor from a file. The editor is pre-loaded with the built-in sample program on first launch.

Right panel (output tabs). A Notebook widget presents six tabs, each backed by a scrollable read-only Text widget with both horizontal and vertical scrollbars:

- *Execution Output* — the `print()` values produced by stage 5, formatted as `value : Type`. The type label is determined at runtime by inspecting the Python type of the

evaluated value; because the type checker guarantees correctness before execution, this always agrees with the statically inferred type;

- *Tokens* — the token stream from stage 1, one token per line with type, lexeme, and source position;
- *Parse Tree* — a grammar-oriented ASCII branch rendering of stage 2 using non-terminal labels such as `statement`, `assignment`, `expr`, `comparison`, `additive`, `term`, and `factor`;
- *AST* — the indented executable tree from stage 3;
- *Type Table* — the symbol table from stage 4 showing each name, kind, inferred type, initialisation state, and parameter list;
- *Errors* — populated only when a pipeline stage fails; shows the error message (e.g. `[line 2, col 5] TypeError: ...`). On a successful run this tab is cleared automatically.

Error handling. If any pipeline stage raises an exception, the error message is written exclusively to the *Errors* tab and the remaining tabs are cleared. No modal dialogs are used, so the user can edit and rerun without dismissing a popup.

Keyboard shortcuts. F5 and Ctrl+Enter are both bound to the *Run* action, allowing rapid edit-run cycles without reaching for the mouse.

6.6.1 Screenshots

Figures 6.2 and 6.3 show the desktop workbench and CLI output.

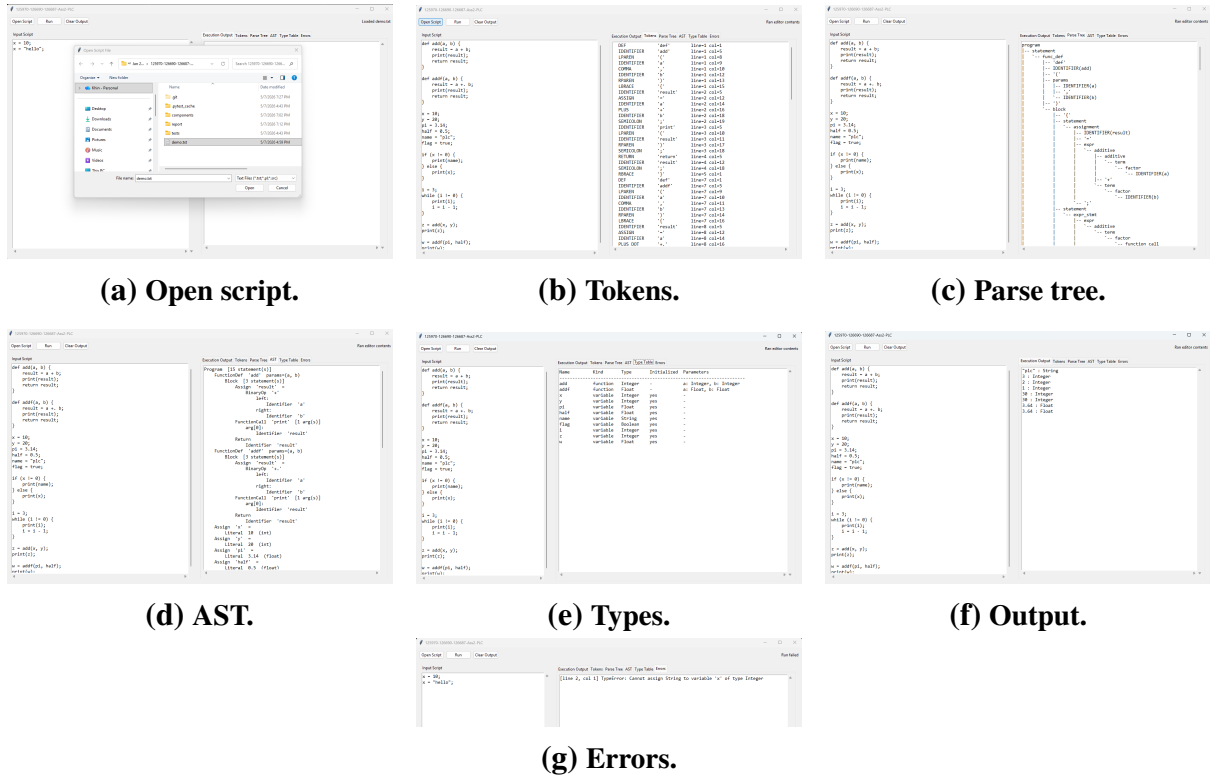


Figure 6.2. Desktop workbench tabs.

6.7 Command-line interface

The CLI runner (`main.py`) accepts an optional script path; without one it runs the built-in sample and prints five labelled stages to stdout. Errors are reported cleanly to stderr: if the specified file does not exist, the runner prints `ERROR: File not found: <path>` and exits with code 1. If any pipeline stage raises a language error (`LexerError`, `ParseError`, `TypeCheckError`, or `InterpreterError`), the message is printed as `ERROR: [line X, col Y] <ErrorType>: <detail>` and the process exits with code 1. In both cases no Python traceback is shown to the user.

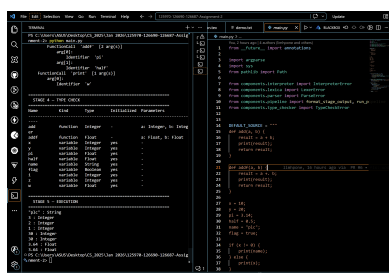
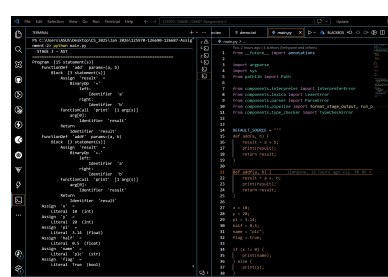
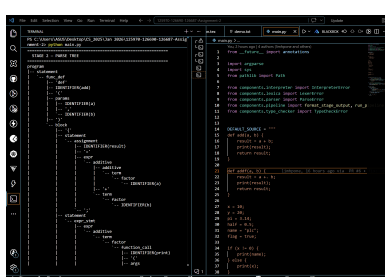
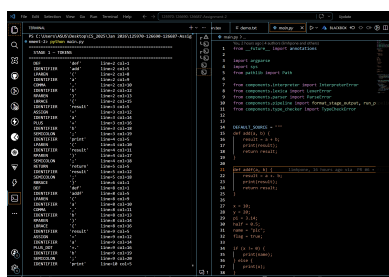


Figure 6.3. CLI stages and error handling.

CHAPTER 7

TESTING AND VERIFICATION

7.1 Manual Testing

Prior to the construction of the automated suite, each language feature was exercised manually by running the full pipeline and inspecting stage outputs. Table 7.1 summarises the categories covered.

Table 7.1. Representative manually tested categories

Category	Program Behaviour	Expected Result
Arithmetic	Evaluate integer and float expressions separately	Correct value and inferred type
If-then-else	Choose one branch from a Boolean condition	Only matching branch runs
While-loop	Print during repeated execution	One output line per iteration
Functions	Infer parameters and return type	Valid call and correct return value
Type mismatch	Reassign incompatible type	Type error before runtime

7.1.1 *Lexer and Symbol Table*

The lexer was verified manually by running the full pipeline and inspecting Stage 1 token output for programs that exercise every token category: integer and float literals, Boolean literals, string literals, the five reserved keywords, two-character comparison operators (`==`, `!=`), single-character operators, and delimiters. Identifier scanning with keyword fallback was confirmed by checking that `true` and `false` are emitted as `BOOL_LITERAL` tokens, the five reserved words (`if`, `else`, `while`, `def`, `return`) as their respective keyword token types, and ordinary names including `print` as `IDENTIFIER` tokens. For example, the source fragment `x = 10 ;` produces the token sequence:

```

1 IDENTIFIER  'x'      line=1 col=1
2 ASSIGN      '='      line=1 col=3
3 INT_LITERAL '10'     line=1 col=5
4 SEMICOLON   ';'      line=1 col=7

```

Error paths were also checked: supplying a bare `!` character raises a `LexerError` with a source position and a message suggesting `!=`, and an unterminated string literal raises a `LexerError` citing the opening line.

The symbol table was verified through the type checker and the Stage 3 output. Duplicate variable definitions were confirmed to raise a `SymbolTableError` before execution. Scope isolation was verified with nested blocks: a variable defined inside an `if` or `while` body is not visible in the outer scope after the block ends. The `format_table()` output was checked against the expected column layout (name, kind, type, initialisation status, parameters) for both variable and function symbols.

7.1.2 Parser, Parse Tree, and AST

The parser was verified manually by running the full pipeline and inspecting both the Stage 2 parse-tree output and the Stage 3 AST output. The following cases were checked:

- the parse-tree view contains grammar-oriented labels such as `program`, `statement`, `assignment`, and `expr`, confirming that the system now exposes a separate parse tree rather than only the executable AST,
- arithmetic expressions with mixed precedence (e.g. `x + y * 2`) produce a tree where multiplication is a deeper sub-tree than addition, confirming the `_expr/_additive/_term/_factor` precedence encoding,
- assignment versus expression statement dispatch: `x = 5`; produces an `Assign` node while `add(1,2)`; produces a `FunctionCall` node at the statement level, confirming the IDENTIFIER+ASSIGN lookahead,
- `if` with and without an `else` clause: the resulting `If` node has a non-null `else_block` only when the clause is present,
- function definitions store only parameter names in the `FunctionDef` node; the AST printer output was inspected to confirm no type annotations are present at this stage,
- string literals in `Literal` nodes have their surrounding double-quotes stripped; `print("hello")` prints `"hello"` : `String` at runtime (the interpreter re-adds the quotes when formatting string output),

- parse errors: missing semicolons, unmatched parentheses, and unexpected tokens all produce `[line X, col Y] ParseError`: messages with correct source positions.

7.1.3 Arithmetic Expressions

Arithmetic expressions were tested with integer-only and float-only programs to verify operator precedence and strict type separation. The expression `x + y * 2` confirmed that multiplication binds more tightly than addition, and mixing integer and float operands correctly produced a type error, confirming the no-overloading design. The test suite explicitly verifies that mixed-type arithmetic is rejected:

- `1 + 2` — valid Integer arithmetic.
- `2.0 +. 3.5` — valid Float arithmetic.
- `5.0 -. 1.5, 2.0 *. 3.0, 9.0 /. 4.0` — valid Float arithmetic for the other dot-suffixed operators.
- `1 + 2.5` — `TypeError`: operator `+` requires two Integer operands.
- `2 +. 3` — `TypeError`: operator `+.` requires two Float operands.
- `10 / 0` and `10.0 /. 0.0` — `InterpreterError`: division by zero, raised at runtime.
- `1.0 +. 2.0 *. 3.0` — evaluates to `7.0` confirming `*.` binds more tightly than `+.`

These rejection tests are not incidental — they are the primary evidence that the language satisfies the specification’s no-overloading requirement. In a promotion-based language, the expressions `1 + 2.5` and `2 +. 3` would silently succeed (returning `3.5 : Float` and `5.0 : Float` respectively). The fact that both are rejected as `TypeError`s demonstrates that each operator token has a single, fixed type signature and that no implicit type coercion occurs. The test suite contains 7 dedicated type-error tests for mixed-type and wrong-type operator usage (see Table 7.3, `TypeCheckerTests`), ensuring that the no-overloading guarantee is verified mechanically rather than by assertion alone.

7.1.4 Boolean Expressions

Boolean expressions were tested inside `if` and `while` conditions. The type checker correctly enforced that both operands of `==` and `!=` must be arithmetic; supplying a `Boolean` or `String` operand produced a source-positioned `TypeCheckError` before any execution.

7.1.5 Assignment Statements

Assignments were tested for both first-use type inference and reassignment consistency. Assigning 5 to a variable and subsequently assigning "hello" to the same variable correctly produced a type error, confirming that the inferred type is fixed after first assignment.

7.1.6 If-Then-Else

Conditional programs were constructed to verify that exactly one branch executes for a given condition and that neither branch is entered when the condition evaluates to the opposite truth value. Non-Boolean conditions were also supplied and confirmed to be rejected by the type checker before execution.

7.1.7 While-Loop

While-loops were tested with an integer countdown from three to zero. The results confirmed that the loop body executes once per iteration and terminates when the condition becomes false, with `print()` producing one output line per pass rather than accumulating output until loop exit.

7.1.8 Functions

Function tests covered value parameter passing, local scope execution, and return value propagation. The type checker correctly inferred parameter types from the first call site and enforced that signature for all subsequent calls to the same function.

7.1.9 Print

The built-in `print()` function was tested with all supported runtime types. The output includes the printed value together with its type, such as `3.14 : Float` and `"plc" : String`.

7.1.10 Unary Minus

The unary minus operator was verified for integer literals (`x = -5`), float literals (`y = -. 3.14`), negated float variables (`y = -. x`), negated integer variables (`y = -x`), and negation inside a larger expression (`x = 3 + -2`). In each case the type checker assigned the correct type and the interpreter produced the expected value. Note that float negation uses the `-.` token, consistent with the Caml Light convention; using plain `-` on a `Float` operand is rejected

by the type checker.

7.2 Type Error Detection

The checker was verified with invalid programs involving incompatible assignment, invalid arithmetic operands, wrong function arguments, and non-Boolean conditions. In each case the system stopped before execution and reported a source-positioned type error. Representative error messages are shown below.

```
1 # Assignment type mismatch
2 [line 2, col 1] TypeError: Cannot assign String to variable 'x' of type
   Integer
3
4 # Non-Boolean if condition
5 [line 1, col 4] TypeError: If condition must have type Boolean
6
7 # Wrong argument type for function
8 [line 5, col 9] TypeError: Argument 1 of 'add' must be Integer, got
   String
```

Table 7.2. Representative negative semantic test cases

Program fragment	Expected result
1 + 2.0	TypeError
x = 5; x = true;	TypeError
if (3) { }	TypeError
print(1, 2)	TypeError
"a" + "b"	TypeError
true == false	TypeError

These negative cases provide direct evidence that the type system enforces the no-overloading and same-type-comparison rules, rather than relying on positive cases alone.

7.3 Automated Test Suite

The automated regression suite is located in `tests/test_pipeline.py` and uses Python's built-in `unittest` framework. The tests are organised into five classes, each targeting a distinct compiler stage.

Table 7.3. Automated test coverage by class

Test class	Stage covered	Tests
<code>LexerTests</code>	Tokenisation (<code>lexica.py</code>)	12
<code>SymbolTableTests</code>	Scoped symbol table (<code>symbol_table.py</code>)	7
<code>ParserTests</code>	Parsing, precedence, parse-tree/AST output	7
<code>TypeCheckerTests</code>	Static type inference (<code>type_checker.py</code>)	22
<code>IntegrationTests</code>	Full pipeline execution	27
Total		75

LexerTests verify that each token category (integer, float, Boolean, string, keywords, identifiers, two-character operators, and the four float-arithmetic operators `+`, `-`, `*`, `/`) is produced correctly, that float literals such as `3.14` are not confused with integer followed by a dot operator, that source line numbers are tracked across newlines, and that illegal characters and unterminated strings raise `LexerError`.

ParserTests verify structural correctness of parsing. The 7 tests cover missing-semicolon and unclosed-brace error reporting, integer operator precedence (`*` before `+`), float operator precedence (`*` before `+`), left-associativity of arithmetic operators, parenthesised expressions overriding precedence, and availability of both the parse-tree and AST renderings in the pipeline result.

SymbolTableTests exercise the symbol table directly, without going through the full pipeline. They cover variable definition and lookup, duplicate definition raising `SymbolTableError` for both variables and functions, parent-scope lookup through `child_scope()`, undefined-symbol lookup error, and the `format_table()` output for both variable and function symbol rows.

TypeCheckerTests verify static type inference and error detection. The 22 tests cover integer and float arithmetic (all four dot-suffixed operators `+`, `-`, `*`, `/` each verified to produce `Float`), integer floor division producing `Integer`, strict type separation (mixed `Integer/Float` rejected for both arithmetic and comparison operators, and integers used with a dot operator rejected), string and boolean type inference, assignment type mismatch, non-Boolean conditions, undefined variables, wrong argument count and type, and two tests for uncalled-function body checking: one confirms that a type error inside a never-called function body is detected, and another confirms that a valid uncalled function raises no error.

IntegrationTests run programs through all four pipeline stages and verify the final output. Cases include unary negation on all numeric types (including `-` applied to a float variable),

division by zero for both `/` and `/.` (each must raise `InterpreterError`), both branches of `if/else`, while-loop countdown, `print()` with every type, value-parameter isolation, nested function calls, comparison results assigned to variables and printed directly, variable reassignment, and scope isolation (variables declared inside an `if` block are not visible outside it).

The tests can be run with:

```
1 # macOS / Linux
2 python3 -m unittest discover -s tests -v
3
4 # Windows
5 python -m unittest discover -s tests -v
```

All 75 tests pass.

7.4 Error Handling

The language system reports errors at four stages:

- lexical errors for malformed characters or unterminated strings,
- parse errors for invalid syntax such as missing semicolons,
- type errors for programs that violate static typing rules,
- runtime errors for division by zero and other execution failures.

All error messages include a source position in the form `[line X, col Y]` to aid debugging.

7.5 Limitations and Future Work

The implemented language intentionally restricts several advanced programming-language features in order to focus on core compiler construction principles and static type inference.

The current implementation does not support recursive functions, arrays, records, polymorphic typing, closures, or garbage collection. Type inference is syntax-directed and deterministic rather than based on full unification algorithms such as Hindley–Milner inference. The execution model is interpreted directly from the AST rather than compiled into bytecode or machine code.

Future extensions may include recursive function support, richer data structures, additional control-flow constructs, optimisation passes, and compilation to a lower-level intermediate representation.

CHAPTER 8

CONCLUSION

This project delivered a complete, statically-typed interpreted language supporting Integer, Float, Boolean, and String types, with arithmetic, comparisons, assignments, if-then-else, while-loops, functions with value parameter passing, and a built-in `print()` function.

The system follows a four-stage pipeline: lexer, parser, type checker, and interpreter. Variable and function types are inferred without explicit declarations; programs that violate type rules are rejected before execution. For uncalled functions, the type checker performs a post-pass to infer parameter types from body usage, ensuring type errors are always caught. A central design decision — following the Caml Light convention of using separate operator tokens for integer and float arithmetic — ensures that the specification’s no-overloading requirement is satisfied in the strongest possible sense: every operator token carries exactly one type signature, and the compiler never inspects operand types to resolve operator meaning. This eliminates not only traditional overloading but also the implicit numeric promotion found in many mainstream languages, resulting in fully deterministic type inference where every expression’s type is derivable from the operator tokens and literal forms alone.

To support clearer correspondence between the formal grammar and accepted programs, the parser stage now exposes both a grammar-oriented parse-tree view and the executable AST used by later stages. The built-in `print()` operation is now implemented as a normal callable built-in rather than a dedicated statement form. The 75 automated tests across five classes — lexical analysis, symbol table, parsing, type checking, and full pipeline — confirm correct behaviour for all language features. Both a command-line runner and a desktop UI are provided.

The project demonstrates how lexical analysis, parsing, semantic analysis, type inference, and interpretation can be integrated into a coherent compiler pipeline with clear separation of responsibilities between stages. The implementation prioritised semantic clarity, deterministic type inference, and modular compiler structure over advanced language features. Future work could extend the language with recursive functions, richer data structures, and code generation.

REFERENCES

- Alfred V. Aho, Monica S. Lam, Ravi Sethi, and Jeffrey D. Ullman. *Compilers: Principles, Techniques, and Tools*. Addison-Wesley, Boston, MA, 2nd edition, 2006.
- Robert Nystrom. *Crafting Interpreters*. Genever Benning, Seattle, WA, 2021.
- Python Software Foundation. Python documentation: dataclasses — data classes. <https://docs.python.org/3/library/dataclasses.html>, 2026a. Accessed: April 2026.
- Python Software Foundation. Python documentation: Tkinter — python interface to tcl/tk. <https://docs.python.org/3/library/tkinter.html>, 2026b. Accessed: April 2026.
- Python Software Foundation. Python documentation: unittest — unit testing framework. <https://docs.python.org/3/library/unittest.html>, 2026c. Accessed: April 2026.

APPENDIX A: SOURCE CODE

The complete source code is available on GitHub:

<https://github.com/The-Token-Trio/125970-126690-126687-Assignment2-PLC>

A video walkthrough of the CLI and GUI is available at:

<https://youtu.be/sR2JRm6eYzM>

The project can be run from the command line or the desktop UI:

```
1 # macOS / Linux
2 python3 main.py
3 python3 ui.py
4 python3 -m unittest discover -s tests -v
5
6 # Windows
7 python main.py
8 python ui.py
9 python -m unittest discover -s tests -v
```

No external Python dependencies are required.

APPENDIX B: TEAM MEMBERS AND CONTRIBUTIONS

Name	Student ID	Contributions
Aye Khin Khin Hpone (Yolanda Lim)	st125970	Static typing and type checking; tree-walking interpreter (assignment, if, while, function, print()); unary minus inference/execution; Desktop UI; CLI; pipeline integration; automated testing (75 tests)
Applegate T. Tun Oo	st126690	Lexer implementation (scanning, tokenisation, line/column tracking, error reporting); token definitions; keywords, operators, identifiers, and literals; symbol table (variable/function/type storage)
Win Htut Naing	st126687	Arithmetic expressions (incl. unary minus); boolean expressions; assignment statements; if-then-else; while-loop; function definitions and calls; print() syntax; parse-tree and AST construction

A.1 Entry Points

```
1 from __future__ import annotations
2
3 import argparse
4 import sys
5 from pathlib import Path
6
7 from components.interpreter import InterpreterError
8 from components.lexica import LexerError
9 from components.parser import ParseError
10 from components.pipeline import format_stage_output, run_pipeline
11 from components.type_checker import TypeCheckError
12
13
14 DEFAULT_SOURCE = """
15 def add(a, b) {
16     result = a + b;
17     print(result);
18     return result;
19 }
20
21 def addf(a, b) {
22     result = a +. b;
23     print(result);
24     return result;
25 }
26
27 x = 10;
28 y = 20;
29 pi = 3.14;
30 half = 0.5;
31 name = "plc";
32 flag = true;
33
34 if (x != 0) {
35     print(name);
36 } else {
37     print(x);
38 }
39
40 i = 3;
41 while (i != 0) {
42     print(i);
43     i = i - 1;
44 }
45
46 z = add(x, y);
47 print(z);
48
49 w = addf(pi, half);
50 print(w);
51 """
```

```

52
53
54 def run_source(source_code: str) -> None:
55     try:
56         result = run_pipeline(source_code)
57         print(format_stage_output(result))
58     except (LexerError, ParseError, TypeCheckError, InterpreterError)
59         as error:
60         print(f"ERROR: {error}", file=sys.stderr)
61         sys.exit(1)
62
63 def _parse_args() -> argparse.Namespace:
64     parser = argparse.ArgumentParser(description="Run the programming
65         language pipeline.")
66     parser.add_argument("script", nargs="?", help="Optional path to a
67         source file to run")
68     return parser.parse_args()
69
70 def main() -> None:
71     args = _parse_args()
72     if args.script:
73         path = Path(args.script)
74         if not path.exists():
75             print(f"ERROR: File not found: {args.script}", file=sys.
76                 stderr)
77             sys.exit(1)
78         source_code = path.read_text(encoding="utf-8")
79     else:
80         source_code = DEFAULT_SOURCE
81     run_source(source_code)
82
83 if __name__ == "__main__":
84     main()

```

Listing 8.1: main.py — command-line entry point

```

1 from __future__ import annotations
2
3 import tkinter as tk
4 from pathlib import Path
5 from tkinter import filedialog, ttk
6
7 from components.pipeline import format_tokens, run_pipeline
8
9
10 DEFAULT_SOURCE = """def add(a, b) {
11     result = a + b;
12     print(result);
13     return result;
14 }

```

```

15
16 def addf(a, b) {
17     result = a +. b;
18     print(result);
19     return result;
20 }
21
22 x = 10;
23 y = 20;
24 pi = 3.14;
25 half = 0.5;
26 name = \"plc\";
27 flag = true;
28
29 if (x != 0) {
30     print(name);
31 } else {
32     print(x);
33 }
34
35 i = 3;
36 while (i != 0) {
37     print(i);
38     i = i - 1;
39 }
40
41 z = add(x, y);
42 print(z);
43
44 w = addf(pi, half);
45 print(w);
46 """
47
48
49 class LanguageWorkbench:
50     def __init__(self) -> None:
51         self.root = tk.Tk()
52         self.root.title("125970-126690-126687-Ass2-PLC")
53         self.root.geometry("1200x760")
54         self.current_file: Path | None = None
55
56         self._build_layout()
57         self.source_text.insert("1.0", DEFAULT_SOURCE)
58         self.root.bind("<F5>", lambda _e: self._run_source())
59         self.root.bind("<Control-Return>", lambda _e: self._run_source
60             ())
61
62     def _build_layout(self) -> None:
63         toolbar = ttk.Frame(self.root, padding=10)
64         toolbar.pack(fill=tk.X)
65
66         ttk.Button(toolbar, text="Open Script", command=self.
67             _open_script).pack(side=tk.LEFT)

```

```

66     ttk.Button(toolbar, text="Run", command=self._run_source).pack(
67         side=tk.LEFT, padx=(8, 0))
68
69     ttk.Button(toolbar, text="Clear Output", command=self.
70         _clear_output).pack(side=tk.LEFT, padx=(8, 0))
71
72     self.status_var = tk.StringVar(value="Ready")
73     ttk.Label(toolbar, textvariable=self.status_var).pack(side=tk.
74         RIGHT)
75
76     body = ttk.Panedwindow(self.root, orient=tk.HORIZONTAL)
77     body.pack(fill=tk.BOTH, expand=True, padx=10, pady=(0, 10))
78
79     left_panel = ttk.Frame(body, padding=8)
80     right_panel = ttk.Frame(body, padding=8)
81     body.add(left_panel, weight=1)
82     body.add(right_panel, weight=1)
83
84     ttk.Label(left_panel, text="Input Script").pack(anchor=tk.W)
85     src_frame = ttk.Frame(left_panel)
86     src_frame.pack(fill=tk.BOTH, expand=True, pady=(6, 0))
87     self.source_text = tk.Text(src_frame, wrap=tk.NONE, font=(
88         "Consolas", 11))
89     src_vsb = ttk.Scrollbar(src_frame, orient=tk.VERTICAL, command=
90         self.source_text.yview)
91     src_hsb = ttk.Scrollbar(src_frame, orient=tk.HORIZONTAL,
92         command=self.source_text.xview)
93     self.source_text.configure(yscrollcommand=src_vsb.set,
94         xscrollcommand=src_hsb.set)
95     src_vsb.pack(side=tk.RIGHT, fill=tk.Y)
96     src_hsb.pack(side=tk.BOTTOM, fill=tk.X)
97     self.source_text.pack(side=tk.LEFT, fill=tk.BOTH, expand=True)
98
99     notebook = ttk.Notebook(right_panel)
100     notebook.pack(fill=tk.BOTH, expand=True)
101
102     self.output_text = self._make_output_tab(notebook, "Execution
103         Output")
104     self.tokens_text = self._make_output_tab(notebook, "Tokens")
105     self.parse_tree_text = self._make_output_tab(notebook, "Parse
106         Tree")
107     self.ast_text = self._make_output_tab(notebook, "AST")
108     self.types_text = self._make_output_tab(notebook, "Type Table")
109     self.error_text = self._make_output_tab(notebook, "Errors")
110
111     def _make_output_tab(self, notebook: ttk.Notebook, title: str) ->
112         tk.Text:
113         frame = ttk.Frame(notebook, padding=8)
114         notebook.add(frame, text=title)
115         text = tk.Text(frame, wrap=tk.NONE, font=("Consolas", 11),
116             state=tk.DISABLED)
117         vsb = ttk.Scrollbar(frame, orient=tk.VERTICAL, command=text.
118             yview)
119         hsb = ttk.Scrollbar(frame, orient=tk.HORIZONTAL, command=text.

```



```

        xview)
107     text.configure(yscrollcommand=vsb.set, xscrollcommand=hsb.set)
108     vsb.pack(side=tk.RIGHT, fill=tk.Y)
109     hsb.pack(side=tk.BOTTOM, fill=tk.X)
110     text.pack(side=tk.LEFT, fill=tk.BOTH, expand=True)
111     return text
112
113     def _open_script(self) -> None:
114         file_path = filedialog.askopenfilename(
115             title="Open Script File",
116             filetypes=[("Text Files", "*.txt *.pl *.src"), ("All Files"
117                 , ".*")],
118         )
119         if not file_path:
120             return
121
122         path = Path(file_path)
123         self.current_file = path
124         self.source_text.delete("1.0", tk.END)
125         self.source_text.insert("1.0", path.read_text(encoding="utf-8")
126             )
127         self.status_var.set(f"Loaded {path.name}")
128
129     def _run_source(self) -> None:
130         source = self.source_text.get("1.0", tk.END)
131         try:
132             result = run_pipeline(source)
133         except Exception as error:
134             self._write_text(self.error_text, str(error))
135             self._write_text(self.output_text, "")
136             self._write_text(self.tokens_text, "")
137             self._write_text(self.parse_tree_text, "")
138             self._write_text(self.ast_text, "")
139             self._write_text(self.types_text, "")
140             self.status_var.set("Run failed")
141             return
142
143         self._write_text(self.output_text, "\n".join(result.outputs) if
144             result.outputs else "(no output)")
145         self._write_text(self.tokens_text, format_tokens(result.tokens)
146             )
147         self._write_text(self.parse_tree_text, result.parse_tree_text)
148         self._write_text(self.ast_text, result.ast_text)
149         self._write_text(self.types_text, result.checked_scope.
150             format_table())
151         self._write_text(self.error_text, "")
152
153         current_label = self.current_file.name if self.current_file is
154             not None else "editor contents"
155         self.status_var.set(f"Ran {current_label}")
156
157     def _clear_output(self) -> None:
158         self._write_text(self.output_text, "")

```

```

153         self._write_text(self.tokens_text, "")
154         self._write_text(self.parse_tree_text, "")
155         self._write_text(self.ast_text, "")
156         self._write_text(self.types_text, "")
157         self._write_text(self.error_text, "")
158         self.status_var.set("Output cleared")
159
160     @staticmethod
161     def _write_text(widget: tk.Text, content: str) -> None:
162         widget.config(state=tk.NORMAL)
163         widget.delete("1.0", tk.END)
164         widget.insert("1.0", content)
165         widget.config(state=tk.DISABLED)
166
167     def run(self) -> None:
168         self.root.mainloop()
169
170
171 if __name__ == "__main__":
172     LanguageWorkbench().run()

```

Listing 8.2: ui.py — Tkinter desktop workbench

A.2 Core Components

```

1 from __future__ import annotations
2
3 from dataclasses import dataclass
4 from enum import Enum, auto
5
6
7 class TokenType(Enum):
8     # Literals
9     INT_LITERAL = auto()
10    FLOAT_LITERAL = auto()
11    BOOL_LITERAL = auto()
12    STRING_LITERAL = auto()
13
14    # Identifier
15    IDENTIFIER = auto()
16
17    # Keywords
18    IF = auto()
19    ELSE = auto()
20    WHILE = auto()
21    DEF = auto()
22    RETURN = auto()
23
24    # Operators
25    PLUS = auto()
26    MINUS = auto()
27    STAR = auto()

```

```

28 SLASH = auto()
29 PLUS_DOT = auto()      # +.  (float addition)
30 MINUS_DOT = auto()     # -.  (float subtraction)
31 STAR_DOT = auto()      # *.  (float multiplication)
32 SLASH_DOT = auto()     # /.  (float division)
33 EQUAL_EQUAL = auto()
34 BANG_EQUAL = auto()
35 ASSIGN = auto()
36
37 # Delimiters
38 LPAREN = auto()
39 RPAREN = auto()
40 LBRACE = auto()
41 RBRACE = auto()
42 COMMA = auto()
43 SEMICOLON = auto()
44
45 EOF = auto()
46
47
48 @dataclass(frozen=True)
49 class Token:
50     token_type: TokenType
51     lexeme: str
52     line: int
53     column: int

```

Listing 8.3: components/tokens.py — token type enumeration and Token dataclass

```

1 from __future__ import annotations
2
3 from typing import Iterator
4
5 from components.tokens import Token, TokenType
6
7
8 KEYWORDS: dict[str, TokenType] = {
9     "if": TokenType.IF,
10    "else": TokenType.ELSE,
11    "while": TokenType.WHILE,
12    "def": TokenType.DEF,
13    "return": TokenType.RETURN,
14    "true": TokenType.BOOL_LITERAL,
15    "false": TokenType.BOOL_LITERAL,
16 }
17
18
19 # Arithmetic operators that may have a trailing '.' for the float
    variant
20 _ARITH_DOT: dict[str, tuple[TokenType, TokenType]] = {
21     "+": (TokenType.PLUS, TokenType.PLUS_DOT),
22     "-": (TokenType.MINUS, TokenType.MINUS_DOT),
23     "*": (TokenType.STAR, TokenType.STAR_DOT),

```

```

24     "/": (TokenType.SLASH,      TokenType.SLASH_DOT),
25 }
26
27 SINGLE_CHAR_TOKENS: dict[str, TokenType] = {
28     "=": TokenType.ASSIGN,
29     "(": TokenType.LPAREN,
30     ")": TokenType.RPAREN,
31     "{": TokenType.LBRACE,
32     "}": TokenType.RBRACE,
33     ",": TokenType.COMMA,
34     ";": TokenType.SEMICOLON,
35 }
36
37
38 class LexerError(Exception):
39     pass
40
41
42 class Lexer:
43     def __init__(self, source: str) -> None:
44         self.source = source
45         self.start = 0
46         self.current = 0
47         self.line = 1
48         self.column = 1
49         self.token_column = 1
50
51     def tokenize(self) -> list[Token]:
52         tokens: list[Token] = []
53         while not self._is_at_end():
54             self.start = self.current
55             self.token_column = self.column
56             token = self._scan_token()
57             if token is not None:
58                 tokens.append(token)
59         tokens.append(Token(TokenType.EOF, "", self.line, self.column))
60         return tokens
61
62     def _scan_token(self) -> Token | None:
63         char = self._advance()
64
65         # Whitespace handling
66         if char in {" ", "\t", "\r"}:
67             return None
68         if char == "\n":
69             self.line += 1
70             self.column = 1
71             return None
72
73         # Two-char operators
74         if char == "=":
75             if self._match("="):
76                 return self._make_token(TokenType.EQUAL_EQUAL)

```

```

77         return self._make_token(TokenType.ASSIGN)
78     if char == "!":
79         if self._match("="):
80             return self._make_token(TokenType.BANG_EQUAL)
81             raise LexerError(self._error_message("Unexpected '!' . Did
              you mean '!= ' ?"))
82
83     # Arithmetic operators: + - * / (with optional trailing '.'
      for float variant)
84     arith = _ARITH_DOT.get(char)
85     if arith is not None:
86         int_tok, float_tok = arith
87         # Unary-minus lookahead: '-' followed by a digit is NOT
          '-' even if
88         # next char is '.' -- we still need to check char after '.'
          is not digit
89         # (to avoid consuming the '.' of a float literal like 3.14)
90         # Rule: X. is a float operator only when the char after '.'
          is NOT a digit.
91         if self._peek() == "." and not self._peek_next().isdigit():
92             self._advance() # consume '.'
93             return self._make_token(float_tok)
94         return self._make_token(int_tok)
95
96     # Single-char operators/delimiters
97     single_char_token = SINGLE_CHAR_TOKENS.get(char)
98     if single_char_token is not None:
99         return self._make_token(single_char_token)
100
101     if char == '"':
102         return self._string()
103
104     if char.isdigit():
105         return self._number()
106
107     if self._is_identifier_start(char):
108         return self._identifier()
109
110     raise LexerError(self._error_message(f"Unexpected character: {
      char!r}"))
111
112     def _string(self) -> Token:
113         while self._peek() != '"' and not self._is_at_end():
114             if self._peek() == "\n":
115                 self.line += 1
116                 self.column = 1
117             self._advance()
118
119         if self._is_at_end():
120             raise LexerError(self._error_message("Unterminated string
              literal"))
121

```

```

122     # Closing quote
123     self._advance()
124     return self._make_token(TokenType.STRING_LITERAL)
125
126 def _number(self) -> Token:
127     while self._peek().isdigit():
128         self._advance()
129
130     is_float = False
131     if self._peek() == "." and self._peek_next().isdigit():
132         is_float = True
133         self._advance()
134         while self._peek().isdigit():
135             self._advance()
136
137     if is_float:
138         return self._make_token(TokenType.FLOAT_LITERAL)
139     return self._make_token(TokenType.INT_LITERAL)
140
141 def _identifier(self) -> Token:
142     while self._is_identifier_part(self._peek()):
143         self._advance()
144
145     lexeme = self._current_lexeme()
146     token_type = KEYWORDS.get(lexeme, TokenType.IDENTIFIER)
147     return self._make_token(token_type)
148
149 def _current_lexeme(self) -> str:
150     return self.source[self.start : self.current]
151
152 def _make_token(self, token_type: TokenType) -> Token:
153     return Token(token_type, self._current_lexeme(), self.line,
154                 self.token_column)
155
156 def _advance(self) -> str:
157     char = self.source[self.current]
158     self.current += 1
159     self.column += 1
160     return char
161
162 def _match(self, expected: str) -> bool:
163     if self._is_at_end():
164         return False
165     if self.source[self.current] != expected:
166         return False
167
168     self.current += 1
169     self.column += 1
170     return True
171
172 def _peek(self) -> str:
173     if self._is_at_end():
174         return "\0"

```

```

174         return self.source[self.current]
175
176     def _peek_next(self) -> str:
177         if self.current + 1 >= len(self.source):
178             return "\0"
179         return self.source[self.current + 1]
180
181     def _is_at_end(self) -> bool:
182         return self.current >= len(self.source)
183
184     @staticmethod
185     def _is_identifier_start(char: str) -> bool:
186         return char.isalpha() or char == "_"
187
188     @staticmethod
189     def _is_identifier_part(char: str) -> bool:
190         return char.isalnum() or char == "_"
191
192     def _error_message(self, message: str) -> str:
193         return f"[line {self.line}, col {self.token_column}] LexerError
194             : {message}"
195
196 def iter_tokens(source: str) -> Iterator[Token]:
197     yield from Lexer(source).tokenize()

```

Listing 8.4: components/lexica.py — lexical analyser

```

1 from __future__ import annotations
2
3 from dataclasses import dataclass, field
4 from typing import Optional
5
6
7 #
8     -----
9
10 # Base
11 #
12     -----
13
14 @dataclass
15 class ASTNode:
16     """Base class for every AST node. Carries source location for
17         diagnostics."""
18     line: int = field(default=0, kw_only=True)
19     column: int = field(default=0, kw_only=True)
20
21 #
22     -----

```

```

19 # Expressions
20 #
21 -----
22 @dataclass
23 class Literal(ASTNode):
24     """An integer, float, boolean, or string literal.
25
26     Examples: 42    3.14    true    "hello"
27     """
28     value: int | float | bool | str
29
30
31 @dataclass
32 class Identifier(ASTNode):
33     """A variable or function name used as an expression.
34
35     Example: x
36     """
37     name: str
38
39
40 @dataclass
41 class BinaryOp(ASTNode):
42     """An arithmetic or comparison binary operation.
43
44     op is one of: '+' '-' '*' '/' '+.' '-.' '*.' '/.' '==' '!='
45     Example: a + b    x == 0    a +. b
46     """
47     op: str
48     left: ASTNode
49     right: ASTNode
50
51
52 @dataclass
53 class FunctionCall(ASTNode):
54     """A call to a user-defined function.
55
56     Example: add(1, 2)
57     """
58     name: str
59     args: list[ASTNode] = field(default_factory=list)
60
61
62 #
63 -----
64 # Statements
65 #
66 -----

```



```

66 @dataclass
67 class Assign(ASTNode):
68     """Variable assignment (first use infers type; later uses must
69         match).
70
71     Example:  x = 10;
72     """
73     name: str
74     value: ASTNode
75
76 @dataclass
77 class Return(ASTNode):
78     """Return statement inside a function body.
79
80     Example:  return result;
81     """
82     expr: ASTNode
83
84
85 @dataclass
86 class Block(ASTNode):
87     """A brace-enclosed sequence of statements.
88
89     Example:  { x = 1; print(x); }
90     """
91     statements: list[ASTNode] = field(default_factory=list)
92
93
94 @dataclass
95 class If(ASTNode):
96     """If / optional else control flow.
97
98     Example:  if (x != 0) { print(x); } else { print(0); }
99     """
100     condition: ASTNode
101     then_block: Block
102     else_block: Optional[Block]
103
104
105 @dataclass
106 class While(ASTNode):
107     """While loop.
108
109     Example:  while (x != 0) { x = x - 1; }
110     """
111     condition: ASTNode
112     body: Block
113
114
115 @dataclass
116 class FunctionDef(ASTNode):
117     """Function definition.

```

```

118
119     Example:  def add(a, b) { return a + b; }
120
121     Note: parameter types are unknown at parse time -- Member 3's type
122           checker
123           will infer them from usage. We store only the parameter names here.
124           """
125     name: str
126     params: list[str]
127     body: Block
128
129 #
130
131 # Top-level
132
133 #
134
135 @dataclass
136 class Program(ASTNode):
137     """Root node: the entire source file as a list of statements."""
138     statements: list[ASTNode] = field(default_factory=list)

```

Listing 8.5: components/ast_nodes.py — AST node definitions

```

1  from __future__ import annotations
2
3  from typing import Optional
4
5  from components.tokens import Token, TokenType
6  from components.ast_nodes import (
7      ASTNode,
8      Program,
9      Block,
10     Assign,
11     If,
12     While,
13     FunctionDef,
14     FunctionCall,
15     Return,
16     BinaryOp,
17     Literal,
18     Identifier,
19 )
20
21 #
22
23 # Parser error
24 #

```

```

25
26 class ParseError(Exception):
27     pass
28
29
30 #
31
32 # Parser
33
34 class Parser:
35     """Recursive-descent parser.
36
37     Usage:
38         tokens = Lexer(source).tokenize()
39         tree    = Parser(tokens).parse()    # returns Program node
40     """
41
42     def __init__(self, tokens: list[Token]) -> None:
43         self._tokens = tokens
44         self._pos = 0
45
46     #
47
48     # Public entry point
49
50     def parse(self) -> Program:
51         """Parse the full token stream and return the root Program node
52         ."""
53         line, col = self._peek().line, self._peek().column
54         statements: list[ASTNode] = []
55         while not self._is_at_end():
56             statements.append(self._statement())
57         return Program(statements=statements, line=line, column=col)
58
59     #
60
61     # Statements
62
63     def _statement(self) -> ASTNode:

```

```

63         """Dispatch to the correct statement parser based on the
        current token."""
64     tok = self._peek()
65
66     if tok.token_type == TokenType.DEF:
67         return self._func_def()
68
69     if tok.token_type == TokenType.IF:
70         return self._if_stmt()
71
72     if tok.token_type == TokenType.WHILE:
73         return self._while_stmt()
74
75     if tok.token_type == TokenType.RETURN:
76         return self._return_stmt()
77
78     # assignment: IDENTIFIER "=" expr ";"
79     if (
80         tok.token_type == TokenType.IDENTIFIER
81         and self._peek_next().token_type == TokenType.ASSIGN
82     ):
83         return self._assignment()
84
85     # fallback: bare expression statement expr ";"
86     node = self._expr()
87     self._expect(TokenType.SEMICOLON, "Expected ';' after
        expression")
88     return node
89
90     def _assignment(self) -> Assign:
91         """IDENTIFIER '=' expr ';'"""
92         name_tok = self._advance() #
93         IDENTIFIER # '='
94         self._advance() # '='
95         value = self._expr()
96         self._expect(TokenType.SEMICOLON, "Expected ';' after
        assignment")
97         return Assign(name=name_tok.lexeme, value=value,
        line=name_tok.line, column=name_tok.column)
98
99     def _if_stmt(self) -> If:
100         """'if' '(' expr ')' block ( 'else' block )?"""
101         tok = self._advance() # 'if'
102         self._expect(TokenType.LPAREN, "Expected '(' after 'if'")
103         condition = self._expr()
104         self._expect(TokenType.RPAREN, "Expected ')' after if condition
        ")
105         then_block = self._block()
106         else_block: Optional[Block] = None
107         if self._check(TokenType.ELSE):
108             self._advance() # 'else'
109             else_block = self._block()

```

```

110         return If(condition=condition, then_block=then_block,
111                    else_block=else_block,
112                    line=tok.line, column=tok.column)
113
114     def _while_stmt(self) -> While:
115         """'while' '(' expr ')' block"""
116         tok = self._advance() # '
117         while'
118         self._expect(TokenType.LPAREN, "Expected '(' after 'while'")
119         condition = self._expr()
120         self._expect(TokenType.RPAREN, "Expected ')' after while
121                        condition")
122         body = self._block()
123         return While(condition=condition, body=body,
124                      line=tok.line, column=tok.column)
125
126     def _func_def(self) -> FunctionDef:
127         """'def' IDENTIFIER '(' params? ')' block"""
128         tok = self._advance() # 'def'
129         name_tok = self._expect(TokenType.IDENTIFIER, "Expected
130                                function name after 'def'")
131         self._expect(TokenType.LPAREN, "Expected '(' after function
132                                name")
133         params = self._params()
134         self._expect(TokenType.RPAREN, "Expected ')' after parameters")
135         body = self._block()
136         return FunctionDef(name=name_tok.lexeme, params=params, body=
137                            body,
138                            line=tok.line, column=tok.column)
139
140     def _return_stmt(self) -> Return:
141         """'return' expr ';'"""
142         tok = self._advance() # '
143         return'
144         expr = self._expr()
145         self._expect(TokenType.SEMICOLON, "Expected ';' after return
146                                value")
147         return Return(expr=expr, line=tok.line, column=tok.column)
148
149     def _block(self) -> Block:
150         """'{', statement* '}'"""
151         tok = self._expect(TokenType.LBRACE, "Expected '{'")
152         statements: list[ASTNode] = []
153         while not self._check(TokenType.RBRACE) and not self._is_at_end
154                ():
155             statements.append(self._statement())
156         self._expect(TokenType.RBRACE, "Expected '}' to close block")
157         return Block(statements=statements, line=tok.line, column=tok.
158                     column)

```

```

151 # Parameters (function definition)
152 #
153 -----
154 def _params(self) -> list[str]:
155     """IDENTIFIER (',' IDENTIFIER)* -- returns list of param names.
156     """
157     params: list[str] = []
158     if self._check(TokenType.IDENTIFIER):
159         params.append(self._advance().lexeme)
160         while self._check(TokenType.COMMA):
161             self._advance() # ','
162             tok = self._expect(TokenType.IDENTIFIER, "Expected
163                 parameter name after ','")
164             params.append(tok.lexeme)
165         return params
166
167 #
168 -----
169 # Expressions (comparison is lowest precedence, then additive,
170 then term)
171 #
172 -----
173
174 def _expr(self) -> ASTNode:
175     """additive (('==' | '!=') additive)? -- non-associative
176     comparison"""
177     node = self._additive()
178     if self._check(TokenType.EQUAL_EQUAL) or self._check(TokenType.
179         BANG_EQUAL):
180         op_tok = self._advance()
181         right = self._additive()
182         node = BinaryOp(op=op_tok.lexeme, left=node, right=right,
183             line=op_tok.line, column=op_tok.column)
184     return node
185
186 def _additive(self) -> ASTNode:
187     """term (('+' | '-' | '+.' | '-.') term)*"""
188     node = self._term()
189     while self._check(TokenType.PLUS) or self._check(TokenType.
190         MINUS) \
191         or self._check(TokenType.PLUS_DOT) or self._check(
192             TokenType.MINUS_DOT):
193         op_tok = self._advance()
194         right = self._term()
195         node = BinaryOp(op=op_tok.lexeme, left=node, right=right,
196             line=op_tok.line, column=op_tok.column)
197     return node
198
199 def _term(self) -> ASTNode:

```

```

191     """factor (('*' | '/' | '*'.' | '/.')
```

```

192 node = self._factor()
```

```

193 while self._check(TokenType.STAR) or self._check(TokenType.
194     SLASH) \
```

```

195         or self._check(TokenType.STAR_DOT) or self._check(
196             TokenType.SLASH_DOT):
```

```

195 op_tok = self._advance()
```

```

196 right = self._factor()
```

```

197 node = BinaryOp(op=op_tok.lexeme, left=node, right=right,
198     line=op_tok.line, column=op_tok.column)
```

```

199 return node
```

```

200
201 def _factor(self) -> ASTNode:
```

```

202     """
```

```

203     '-' factor (unary negation, desugared to 0 - factor)
```

```

204     | INT_LITERAL | FLOAT_LITERAL | STRING_LITERAL | BOOL_LITERAL
```

```

205     | IDENTIFIER ( '(' args? ')' )?
```

```

206     | '(' expr ')'
```

```

207     """
```

```

208 tok = self._peek()
```

```

209
210 # Unary minus '-' factor (integer negation, desugared to 0 -
211     factor)
```

```

211 if tok.token_type == TokenType.MINUS:
```

```

212     op_tok = self._advance()
```

```

213     operand = self._factor()
```

```

214     zero = Literal(value=0, line=op_tok.line, column=op_tok.
215         column)
```

```

215     return BinaryOp(op="-", left=zero, right=operand,
```

```

216         line=op_tok.line, column=op_tok.column)
```

```

217
218 # Unary minus-dot '-.' factor (float negation, desugared to
219     0.0 -. factor)
```

```

219 if tok.token_type == TokenType.MINUS_DOT:
```

```

220     op_tok = self._advance()
```

```

221     operand = self._factor()
```

```

222     zero_f = Literal(value=0.0, line=op_tok.line, column=op_tok.
223         .column)
```

```

223     return BinaryOp(op="-.", left=zero_f, right=operand,
```

```

224         line=op_tok.line, column=op_tok.column)
```

```

225
226 # Integer literal
```

```

227 if tok.token_type == TokenType.INT_LITERAL:
```

```

228     self._advance()
```

```

229     return Literal(value=int(tok.lexeme), line=tok.line, column
230         =tok.column)
```

```

230
231 # Float literal
```

```

232 if tok.token_type == TokenType.FLOAT_LITERAL:
```

```

233     self._advance()
```

```

234     return Literal(value=float(tok.lexeme), line=tok.line,
235         column=tok.column)
```

```

236     # Boolean literal
237     if tok.token_type == TokenType.BOOL_LITERAL:
238         self._advance()
239         return Literal(value=(tok.lexeme == "true"), line=tok.line,
240                        column=tok.column)
241
242     # String literal (keep the surrounding quotes stripped)
243     if tok.token_type == TokenType.STRING_LITERAL:
244         self._advance()
245         return Literal(value=tok.lexeme[1:-1], line=tok.line,
246                        column=tok.column)
247
248     # Identifier -- could be a plain variable or a function call
249     if tok.token_type == TokenType.IDENTIFIER:
250         self._advance()
251         if self._check(TokenType.LPAREN):
252             # function call
253             self._advance()
254             args = self._args()
255             self._expect(TokenType.RPAREN, "Expected ')' after
256                           function arguments")
257             return FunctionCall(name=tok.lexeme, args=args,
258                                line=tok.line, column=tok.column)
259         return Identifier(name=tok.lexeme, line=tok.line, column=
260                           tok.column)
261
262     # Grouped expression '(' expr ')'
263     if tok.token_type == TokenType.LPAREN:
264         self._advance()
265         node = self._expr()
266         self._expect(TokenType.RPAREN, "Expected ')' to close
267                           grouped expression")
268         return node
269
270     raise ParseError(self._error_at(tok, f"Unexpected token: {tok.
271                                         lexeme!r}"))
272
273     #
274     -----
275
276     # Arguments (function call)
277     #
278     -----
279
280     def _args(self) -> list[ASTNode]:
281         """expr (',' expr)*"""
282         args: list[ASTNode] = []
283         if not self._check(TokenType.RPAREN):
284             args.append(self._expr())
285             while self._check(TokenType.COMMA):
286                 self._advance()
287                 args.append(self._expr())

```



```

279     return args
280
281     #
282     -----
283
284     # Token navigation helpers
285     #
286     -----
287
288     def _peek(self) -> Token:
289         return self._tokens[self._pos]
290
291     def _peek_next(self) -> Token:
292         if self._pos + 1 < len(self._tokens):
293             return self._tokens[self._pos + 1]
294         return self._tokens[-1] # EOF
295
296     def _advance(self) -> Token:
297         tok = self._tokens[self._pos]
298         if not self._is_at_end():
299             self._pos += 1
300         return tok
301
302     def _check(self, token_type: TokenType) -> bool:
303         return self._peek().token_type == token_type
304
305     def _is_at_end(self) -> bool:
306         return self._peek().token_type == TokenType.EOF
307
308     def _expect(self, token_type: TokenType, message: str) -> Token:
309         if self._check(token_type):
310             return self._advance()
311         raise ParseError(self._error_at(self._peek(), message))
312
313     def _error_at(self, tok: Token, message: str) -> str:
314         return f"[line {tok.line}, col {tok.column}] ParseError: {message}"

```

Listing 8.6: components/parser.py — recursive-descent parser

```

1 from __future__ import annotations
2
3 from dataclasses import dataclass, field
4
5 from components.ast_nodes import (
6     ASTNode,
7     Assign,
8     BinaryOp,
9     Block,
10    FunctionCall,
11    FunctionDef,
12    Identifier,

```

```

13     If,
14     Literal,
15     Program,
16     Return,
17     While,
18 )
19
20
21 @dataclass
22 class ParseTreeNode:
23     label: str
24     children: list["ParseTreeNode"] = field(default_factory=list)
25
26
27 class ParseTreePrinter:
28     """Renders a textbook-style concrete parse tree using ASCII
29     branches."""
30
31     def print(self, node: Program) -> str:
32         root = self._program_node(node)
33         lines: list[str] = []
34         self._render(root, lines, prefix="", is_last=True, is_root=True)
35         return "\n".join(lines)
36
37     def _render(
38         self,
39         node: ParseTreeNode,
40         lines: list[str],
41         prefix: str,
42         is_last: bool,
43         is_root: bool = False,
44     ) -> None:
45         if is_root:
46             lines.append(node.label)
47         else:
48             branch = "'-- " if is_last else "|-- "
49             lines.append(f"{prefix}{branch}{node.label}")
50
51         child_prefix = prefix + ("    " if is_last else "|    ")
52         for index, child in enumerate(node.children):
53             self._render(
54                 child,
55                 lines,
56                 prefix=child_prefix if not is_root else "",
57                 is_last=index == len(node.children) - 1,
58             )
59
60     def _program_node(self, node: Program) -> ParseTreeNode:
61         children = [self._statement_node(statement) for statement in
62                     node.statements]
63         children.append(ParseTreeNode("EOF"))
64         return ParseTreeNode("program", children)

```

```

63
64 def _statement_node(self, node: ASTNode) -> ParseTreeNode:
65     if isinstance(node, Assign):
66         return ParseTreeNode("statement", [self._assignment_node(
67             node)])
68     if isinstance(node, If):
69         return ParseTreeNode("statement", [self._if_node(node)])
70     if isinstance(node, While):
71         return ParseTreeNode("statement", [self._while_node(node)])
72     if isinstance(node, FunctionDef):
73         return ParseTreeNode("statement", [self._function_def_node(
74             node)])
75     if isinstance(node, Return):
76         return ParseTreeNode("statement", [self._return_node(node)
77             ])
78     return ParseTreeNode(
79         "statement",
80         [
81             ParseTreeNode(
82                 "expr_stmt",
83                 [self._expr_node(node), ParseTreeNode("';'")],
84             )
85         ],
86     )
87
88 def _assignment_node(self, node: Assign) -> ParseTreeNode:
89     return ParseTreeNode(
90         "assignment",
91         [
92             ParseTreeNode(f"IDENTIFIER({node.name})",
93                 ParseTreeNode("'= '"),
94                 self._expr_node(node.value),
95                 ParseTreeNode("';'"),
96             )
97         ],
98     )
99
100 def _if_node(self, node: If) -> ParseTreeNode:
101     children = [
102         ParseTreeNode("'if'"),
103         ParseTreeNode("'('"),
104         self._expr_node(node.condition),
105         ParseTreeNode("')'"),
106         self._block_node(node.then_block),
107     ]
108     if node.else_block is not None:
109         children.extend([ParseTreeNode("'else'"), self._block_node(
110             node.else_block)])
111     return ParseTreeNode("if_stmt", children)
112
113 def _while_node(self, node: While) -> ParseTreeNode:
114     return ParseTreeNode(
115         "while_stmt",
116         [

```

```

112         ParseTreeNode("'while'"),
113         ParseTreeNode("'('"),
114         self._expr_node(node.condition),
115         ParseTreeNode("')'"),
116         self._block_node(node.body),
117     ],
118 )
119
120 def _function_def_node(self, node: FunctionDef) -> ParseTreeNode:
121     return ParseTreeNode(
122         "func_def",
123         [
124             ParseTreeNode("'def'"),
125             ParseTreeNode(f"IDENTIFIER({node.name})"),
126             ParseTreeNode("'('"),
127             self._params_node(node.params),
128             ParseTreeNode("')'"),
129             self._block_node(node.body),
130         ],
131     )
132
133 def _params_node(self, params: list[str]) -> ParseTreeNode:
134     if not params:
135         return ParseTreeNode("params", [ParseTreeNode("epsilon")])
136
137     children: list[ParseTreeNode] = []
138     for index, param in enumerate(params):
139         children.append(ParseTreeNode(f"IDENTIFIER({param})"))
140         if index < len(params) - 1:
141             children.append(ParseTreeNode("','"))
142     return ParseTreeNode("params", children)
143
144 def _return_node(self, node: Return) -> ParseTreeNode:
145     return ParseTreeNode(
146         "return_stmt",
147         [
148             ParseTreeNode("'return'"),
149             self._expr_node(node.expr),
150             ParseTreeNode("';'"),
151         ],
152     )
153
154 def _block_node(self, node: Block) -> ParseTreeNode:
155     children = [ParseTreeNode("'{'")]
156     children.extend(self._statement_node(statement) for statement
157                     in node.statements)
158     children.append(ParseTreeNode("'}'"))
159     return ParseTreeNode("block", children)
160
161 def _expr_node(self, node: ASTNode) -> ParseTreeNode:
162     if isinstance(node, BinaryOp) and node.op in {"==", "!="}:
163         child = ParseTreeNode(
164             "comparison",

```

```

164         [
165             self._additive_node(node.left),
166             ParseTreeNode(f" '{node.op}'", ),
167             self._additive_node(node.right),
168         ],
169     )
170     return ParseTreeNode("expr", [child])
171 return ParseTreeNode("expr", [self._additive_node(node)])
172
173 def _additive_node(self, node: ASTNode) -> ParseTreeNode:
174     if isinstance(node, BinaryOp) and node.op in {"+", "-", "+.", "-."}:
175         return ParseTreeNode(
176             "additive",
177             [
178                 self._additive_node(node.left),
179                 ParseTreeNode(f" '{node.op}'", ),
180                 self._term_node(node.right),
181             ],
182         )
183     return ParseTreeNode("additive", [self._term_node(node)])
184
185 def _term_node(self, node: ASTNode) -> ParseTreeNode:
186     if isinstance(node, BinaryOp) and node.op in {"*", "/", "*.", "/."}:
187         return ParseTreeNode(
188             "term",
189             [
190                 self._term_node(node.left),
191                 ParseTreeNode(f" '{node.op}'", ),
192                 self._factor_node(node.right),
193             ],
194         )
195     return ParseTreeNode("term", [self._factor_node(node)])
196
197 def _factor_node(self, node: ASTNode) -> ParseTreeNode:
198     if self._is_integer_unary_minus(node):
199         binary = node
200         assert isinstance(binary, BinaryOp)
201         return ParseTreeNode("factor", [ParseTreeNode("'-'", ), self._factor_node(binary.right)])
202     if self._is_float_unary_minus(node):
203         binary = node
204         assert isinstance(binary, BinaryOp)
205         return ParseTreeNode("factor", [ParseTreeNode("'-.'", ), self._factor_node(binary.right)])
206     if isinstance(node, Literal):
207         return ParseTreeNode("factor", [ParseTreeNode(self._literal_label(node.value))])
208     if isinstance(node, Identifier):
209         return ParseTreeNode("factor", [ParseTreeNode(f"IDENTIFIER({node.name})")])
210     if isinstance(node, FunctionCall):

```

```

211         return ParseTreeNode("factor", [self._function_call_node(
212             node)])
213     return ParseTreeNode(
214         "factor",
215         [ParseTreeNode("'(',')", self._expr_node(node), ParseTreeNode(
216             "'')'")],
217     )
218
219 def _function_call_node(self, node: FunctionCall) -> ParseTreeNode:
220     return ParseTreeNode(
221         "function_call",
222         [
223             ParseTreeNode(f"IDENTIFIER({node.name})",
224                 ParseTreeNode("'(',')",
225                     self._args_node(node.args),
226                     ParseTreeNode("'')'")),
227         ],
228     )
229
230 def _args_node(self, args: list[ASTNode]) -> ParseTreeNode:
231     if not args:
232         return ParseTreeNode("args", [ParseTreeNode("epsilon")])
233
234     children: list[ParseTreeNode] = []
235     for index, arg in enumerate(args):
236         children.append(self._expr_node(arg))
237         if index < len(args) - 1:
238             children.append(ParseTreeNode("'',''"))
239     return ParseTreeNode("args", children)
240
241 @staticmethod
242 def _is_integer_unary_minus(node: ASTNode) -> bool:
243     return (
244         isinstance(node, BinaryOp)
245         and node.op == "-"
246         and isinstance(node.left, Literal)
247         and node.left.value == 0
248     )
249
250 @staticmethod
251 def _is_float_unary_minus(node: ASTNode) -> bool:
252     return (
253         isinstance(node, BinaryOp)
254         and node.op == "-."
255         and isinstance(node.left, Literal)
256         and node.left.value == 0.0
257     )
258
259 @staticmethod
260 def _literal_label(value: object) -> str:
261     if isinstance(value, bool):
262         return f"BOOL_LITERAL({'true' if value else 'false'})"
263     if isinstance(value, int):

```

```

262         return f"INT_LITERAL({value})"
263     if isinstance(value, float):
264         return f"FLOAT_LITERAL({value})"
265     if isinstance(value, str):
266         return f'String_LITERAL("{value}")'
267     return f"LITERAL({value!r})"

```

Listing 8.7: components/parse_tree_printer.py — parse-tree renderer

```

1  from __future__ import annotations
2
3  from components.ast_nodes import (
4      ASTNode,
5      Program,
6      Block,
7      Assign,
8      If,
9      While,
10     FunctionDef,
11     FunctionCall,
12     Return,
13     BinaryOp,
14     Literal,
15     Identifier,
16 )
17
18
19 class ASTPrinter:
20     """Walks an AST and returns a human-readable, indented string.
21
22     Usage:
23         tree = Parser(tokens).parse()
24         print(ASTPrinter().print(tree))
25
26     Output style -- resembles the original source but annotated with
27     node types,
28     making it easy to verify the tree structure during development and
29     in reports.
30     """
31
32     INDENT = "    " # 4 spaces per level
33
34     def print(self, node: ASTNode) -> str:
35         lines: list[str] = []
36         self._visit(node, lines, level=0)
37         return "\n".join(lines)
38
39     #
40     # -----
41
42     # Dispatcher
43     #
44     # -----

```

```

40
41 def _visit(self, node: ASTNode, lines: list[str], level: int) ->
    None:
42     method_name = "_visit_" + type(node).__name__
43     visitor = getattr(self, method_name, self._visit_unknown)
44     visitor(node, lines, level)
45
46 def _pad(self, level: int) -> str:
47     return self.INDENT * level
48
49 #
    -----
50 # Node visitors
51 #
    -----
52
53 def _visit_Program(self, node: Program, lines: list[str], level:
    int) -> None:
54     lines.append(f"{self._pad(level)}Program [{len(node.statements
        )} statement(s)]")
55     for stmt in node.statements:
56         self._visit(stmt, lines, level + 1)
57
58 def _visit_Block(self, node: Block, lines: list[str], level: int)
    -> None:
59     lines.append(f"{self._pad(level)}Block [{len(node.statements)}
        statement(s)]")
60     for stmt in node.statements:
61         self._visit(stmt, lines, level + 1)
62
63 def _visit_Assign(self, node: Assign, lines: list[str], level: int)
    -> None:
64     lines.append(f"{self._pad(level)}Assign {node.name!r} =")
65     self._visit(node.value, lines, level + 1)
66
67 def _visit_If(self, node: If, lines: list[str], level: int) -> None
    :
68     lines.append(f"{self._pad(level)}If")
69     lines.append(f"{self._pad(level + 1)}condition:")
70     self._visit(node.condition, lines, level + 2)
71     lines.append(f"{self._pad(level + 1)}then:")
72     self._visit(node.then_block, lines, level + 2)
73     if node.else_block is not None:
74         lines.append(f"{self._pad(level + 1)}else:")
75         self._visit(node.else_block, lines, level + 2)
76
77 def _visit_While(self, node: While, lines: list[str], level: int)
    -> None:
78     lines.append(f"{self._pad(level)}While")
79     lines.append(f"{self._pad(level + 1)}condition:")

```



```

80     self._visit(node.condition, lines, level + 2)
81     lines.append(f"{self._pad(level + 1)}body:")
82     self._visit(node.body, lines, level + 2)
83
84     def _visit_FunctionDef(self, node: FunctionDef, lines: list[str],
85                             level: int) -> None:
86         params = ", ".join(node.params) if node.params else "(none)"
87         lines.append(f"{self._pad(level)}FunctionDef {node.name!r}
88                     params=({params})")
89         self._visit(node.body, lines, level + 1)
90
91     def _visit_FunctionCall(self, node: FunctionCall, lines: list[str],
92                             level: int) -> None:
93         lines.append(f"{self._pad(level)}FunctionCall {node.name!r}
94                     [{len(node.args)} arg(s)]")
95         for i, arg in enumerate(node.args):
96             lines.append(f"{self._pad(level + 1)}arg[{i}]:")
97             self._visit(arg, lines, level + 2)
98
99     def _visit_Return(self, node: Return, lines: list[str], level: int)
100         -> None:
101         lines.append(f"{self._pad(level)}Return")
102         self._visit(node.expr, lines, level + 1)
103
104     def _visit_BinaryOp(self, node: BinaryOp, lines: list[str], level:
105         int) -> None:
106         lines.append(f"{self._pad(level)}BinaryOp '{node.op}'")
107         lines.append(f"{self._pad(level + 1)}left:")
108         self._visit(node.left, lines, level + 2)
109         lines.append(f"{self._pad(level + 1)}right:")
110         self._visit(node.right, lines, level + 2)
111
112     def _visit_Literal(self, node: Literal, lines: list[str], level:
113         int) -> None:
114         type_name = type(node.value).__name__
115         lines.append(f"{self._pad(level)}Literal {node.value!r} ({
116                     type_name})")
117
118     def _visit_Identifier(self, node: Identifier, lines: list[str],
119                             level: int) -> None:
120         lines.append(f"{self._pad(level)}Identifier {node.name!r}")
121
122     def _visit_unknown(self, node: ASTNode, lines: list[str], level:
123         int) -> None:
124         lines.append(f"{self._pad(level)}??? Unknown node: {type(node)}.
125                     __name__")

```

Listing 8.8: components/ast_printer.py — AST pretty-printer

```

1 from __future__ import annotations
2
3 from dataclasses import dataclass, field
4 from enum import Enum

```

```

5
6
7 class LanguageType(Enum):
8     INTEGER = "Integer"
9     FLOAT = "Float"
10    BOOLEAN = "Boolean"
11    STRING = "String"
12    VOID = "Void"
13
14
15 @dataclass
16 class Symbol:
17     name: str
18     symbol_type: LanguageType
19
20
21 @dataclass
22 class VariableSymbol(Symbol):
23     initialized: bool = False
24
25
26 @dataclass
27 class FunctionSymbol(Symbol):
28     parameters: list[tuple[str, LanguageType]] = field(default_factory=
29         list)
30
31 class SymbolTableError(Exception):
32     pass
33
34
35 class SymbolTable:
36     def __init__(self, parent: SymbolTable | None = None) -> None:
37         self.parent = parent
38         self._symbols: dict[str, Symbol] = {}
39
40     def define_variable(self, name: str, symbol_type: LanguageType,
41         initialized: bool = False) -> VariableSymbol:
42         if name in self._symbols:
43             raise SymbolTableError(f"Symbol already defined in this
44                 scope: {name}")
45         symbol = VariableSymbol(name=name, symbol_type=symbol_type,
46             initialized=initialized)
47         self._symbols[name] = symbol
48         return symbol
49
50     def define_function(
51         self,
52         name: str,
53         return_type: LanguageType,
54         parameters: list[tuple[str, LanguageType]],
55     ) -> FunctionSymbol:
56         if name in self._symbols:

```

```

54         raise SymbolTableError(f"Symbol already defined in this
55             scope: {name}")
56     symbol = FunctionSymbol(name=name, symbol_type=return_type,
57         parameters=parameters)
58     self._symbols[name] = symbol
59     return symbol
60
61 def lookup(self, name: str) -> Symbol:
62     symbol = self._symbols.get(name)
63     if symbol is not None:
64         return symbol
65     if self.parent is not None:
66         return self.parent.lookup(name)
67     raise SymbolTableError(f"Undefined symbol: {name}")
68
69 def exists_in_current_scope(self, name: str) -> bool:
70     return name in self._symbols
71
72 def child_scope(self) -> SymbolTable:
73     return SymbolTable(parent=self)
74
75 def set_initialized(self, name: str) -> None:
76     symbol = self.lookup(name)
77     if not isinstance(symbol, VariableSymbol):
78         raise SymbolTableError(f"Cannot initialize non-variable
79             symbol: {name}")
80     symbol.initialized = True
81
82 def snapshot(self) -> dict[str, Symbol]:
83     return dict(self._symbols)
84
85 def format_table(self) -> str:
86     lines = [
87         f"{'Name':<12} {'Kind':<10} {'Type':<10} {'Initialized'
88             ':<12} Parameters",
89         "-" * 72,
90     ]
91
92     for name, symbol in self._symbols.items():
93         if isinstance(symbol, FunctionSymbol):
94             params = ", ".join(f"{param_name}: {param_type.value}"
95                 for param_name, param_type in symbol.parameters)
96             lines.append(
97                 f"{name:<12} {'function':<10} {symbol.symbol_type.
98                     value:<10} {'-':<12} {params or '-'}"
99             )
100         elif isinstance(symbol, VariableSymbol):
101             initialized = "yes" if symbol.initialized else "no"
102             lines.append(
103                 f"{name:<12} {'variable':<10} {symbol.symbol_type.
104                     value:<10} {initialized:<12} -"
105             )
106         else:

```

```

100         lines.append(f"{name:<12} {'symbol':<10} {symbol.
101             symbol_type.value:<10} {'-':<12} -")
102
103     return "\n".join(lines)

```

Listing 8.9: components/symbol_table.py — scoped symbol table

```

1  from __future__ import annotations
2
3  from dataclasses import dataclass
4
5  from components.ast_nodes import (
6      ASTNode,
7      Assign,
8      BinaryOp,
9      Block,
10     FunctionCall,
11     FunctionDef,
12     Identifier,
13     If,
14     Literal,
15     Program,
16     Return,
17     While,
18 )
19 from components.symbol_table import FunctionSymbol, LanguageType,
20     SymbolTable, SymbolTableError, VariableSymbol
21
22 class TypeCheckError(Exception):
23     pass
24
25
26 @dataclass
27 class _FunctionState:
28     node: FunctionDef
29     body_checked: bool = False
30
31
32 @dataclass
33 class _FunctionContext:
34     name: str
35     return_type: LanguageType | None = None
36
37
38 class TypeChecker:
39     _BUILTIN_PRINT = "print"
40
41     def __init__(self) -> None:
42         self.global_scope = SymbolTable()
43         self._functions: dict[str, _FunctionState] = {}
44         self._active_calls: list[str] = []
45

```

```

46 def check(self, program: Program) -> SymbolTable:
47     for statement in program.statements:
48         if isinstance(statement, FunctionDef):
49             self._register_function(statement)
50
51     for statement in program.statements:
52         if isinstance(statement, FunctionDef):
53             continue
54         self._check_statement(statement, self.global_scope, None)
55
56     self._check_uncalled_functions()
57     return self.global_scope
58
59 def _register_function(self, node: FunctionDef) -> None:
60     if node.name in self._functions:
61         raise TypeCheckError(self._error_at(node, f"Function '{node
62             .name}' is already defined"))
63
64     placeholder_parameters = [(param_name, LanguageType.VOID) for
65         param_name in node.params]
66     try:
67         self.global_scope.define_function(node.name, LanguageType.
68             VOID, placeholder_parameters)
69     except SymbolTableError as error:
70         raise TypeCheckError(self._error_at(node, str(error))) from
71         error
72     self._functions[node.name] = _FunctionState(node=node)
73
74 def _check_statement(
75     self,
76     node: ASTNode,
77     scope: SymbolTable,
78     function_context: _FunctionContext | None,
79 ) -> None:
80     if isinstance(node, Assign):
81         value_type = self._infer_expr_type(node.value, scope)
82         self._assign_variable_type(scope, node, value_type)
83         return
84
85     if isinstance(node, Return):
86         if function_context is None:
87             raise TypeCheckError(self._error_at(node, "'return' is
88                 only valid inside a function"))
89         expr_type = self._infer_expr_type(node.expr, scope)
90         if function_context.return_type is None:
91             function_context.return_type = expr_type
92         elif function_context.return_type != expr_type:
93             raise TypeCheckError(
94                 self._error_at(
95                     node,
96                     f"Function '{function_context.name}' returns
97                         both {function_context.return_type.value}
98                         and {expr_type.value}",

```

```

92         )
93     )
94     return
95
96     if isinstance(node, If):
97         condition_type = self._infer_expr_type(node.condition,
98             scope)
99         if condition_type != LanguageType.BOOLEAN:
100             raise TypeCheckError(self._error_at(node.condition, "If
101                 condition must have type Boolean"))
102         self._check_block(node.then_block, scope.child_scope(),
103             function_context)
104         if node.else_block is not None:
105             self._check_block(node.else_block, scope.child_scope(),
106                 function_context)
107         return
108
109     if isinstance(node, While):
110         condition_type = self._infer_expr_type(node.condition,
111             scope)
112         if condition_type != LanguageType.BOOLEAN:
113             raise TypeCheckError(self._error_at(node.condition, "
114                 While condition must have type Boolean"))
115         self._check_block(node.body, scope.child_scope(),
116             function_context)
117         return
118
119     if isinstance(node, Block):
120         self._check_block(node, scope.child_scope(),
121             function_context)
122         return
123
124     if isinstance(node, FunctionCall):
125         self._infer_function_call_type(node, scope)
126         return
127
128     raise TypeCheckError(self._error_at(node, f"Unsupported
129         statement node: {type(node).__name__}"))
130
131     def _check_block(
132         self,
133         block: Block,
134         scope: SymbolTable,
135         function_context: _FunctionContext | None,
136     ) -> None:
137         for statement in block.statements:
138             self._check_statement(statement, scope, function_context)
139
140     def _infer_expr_type(self, node: ASTNode, scope: SymbolTable) ->
141         LanguageType:
142         if isinstance(node, Literal):
143             return self._literal_type(node)

```

```

135     if isinstance(node, Identifier):
136         try:
137             symbol = scope.lookup(node.name)
138         except SymbolTableError as error:
139             raise TypeCheckError(self._error_at(node, str(error)))
140             from error
141         if not isinstance(symbol, VariableSymbol):
142             raise TypeCheckError(self._error_at(node, f"'{node.name
143                 }' is not a variable"))
144         return symbol.symbol_type
145
146     if isinstance(node, FunctionCall):
147         return self._infer_function_call_type(node, scope)
148
149     if isinstance(node, BinaryOp):
150         left_type = self._infer_expr_type(node.left, scope)
151         right_type = self._infer_expr_type(node.right, scope)
152         return self._infer_binary_type(node, left_type, right_type)
153
154     raise TypeCheckError(self._error_at(node, f"Unsupported
155         expression node: {type(node).__name__}"))
156
157 def _infer_function_call_type(self, node: FunctionCall, scope:
158     SymbolTable) -> LanguageType:
159     if node.name == self._BUILTIN_PRINT:
160         return self._infer_builtin_print_type(node, scope)
161
162     function_state = self._functions.get(node.name)
163     if function_state is None:
164         raise TypeCheckError(self._error_at(node, f"Undefined
165             function: {node.name}"))
166     if node.name in self._active_calls:
167         raise TypeCheckError(self._error_at(node, f"Recursive
168             function calls are not supported: {node.name}"))
169
170     argument_types = [self._infer_expr_type(argument, scope) for
171         argument in node.args]
172     function_symbol = self._lookup_function_symbol(node)
173
174     if len(argument_types) != len(function_symbol.parameters):
175         raise TypeCheckError(
176             self._error_at(
177                 node,
178                 f"Function '{node.name}' expects {len(
179                     function_symbol.parameters)} argument(s), got {
180                     len(argument_types)}",
181             )
182         )
183
184     declared_parameter_types = [param_type for _, param_type in
185         function_symbol.parameters]
186     if all(param_type == LanguageType.VOID for param_type in
187         declared_parameter_types):

```

```

177         function_symbol.parameters = list(zip(function_state.node.
178             params, argument_types, strict=False))
179     else:
180         for index, ((_ , expected_type), actual_type) in enumerate(
181             zip(function_symbol.parameters, argument_types, strict=
182                 False)):
183             if expected_type != actual_type:
184                 raise TypeCheckError(
185                     self._error_at(
186                         node.args[index],
187                         f"Argument {index + 1} of '{node.name}'
188                             must be {expected_type.value}, got {
189                                 actual_type.value}",
190                     )
191                 )
192
193     if not function_state.body_checked:
194         self._check_function_body(function_state, function_symbol)
195
196     return function_symbol.symbol_type
197
198 def _infer_builtin_print_type(self, node: FunctionCall, scope:
199     SymbolTable) -> LanguageType:
200     if len(node.args) != 1:
201         raise TypeCheckError(
202             self._error_at(node, f"Built-in function 'print'
203                 expects 1 argument, got {len(node.args)}")
204         )
205     self._infer_expr_type(node.args[0], scope)
206     return LanguageType.VOID
207
208 def _check_function_body(self, function_state: _FunctionState,
209     function_symbol: FunctionSymbol) -> None:
210     function_context = _FunctionContext(name=function_state.node.
211         name)
212     function_scope = self.global_scope.child_scope()
213     for parameter_name, parameter_type in function_symbol.
214         parameters:
215         function_scope.define_variable(parameter_name,
216             parameter_type, initialized=True)
217
218     self._active_calls.append(function_state.node.name)
219     try:
220         self._check_block(function_state.node.body, function_scope,
221             function_context)
222     finally:
223         self._active_calls.pop()
224
225     function_symbol.symbol_type = function_context.return_type or
226         LanguageType.VOID
227     function_state.body_checked = True
228
229 def _check_uncalled_functions(self) -> None:

```



```

217     """After all call-site checks, type-check any function whose
218         body was never visited."""
219     for name, state in self._functions.items():
220         if not state.body_checked:
221             try:
222                 function_symbol = self.global_scope.lookup(name)
223             except SymbolTableError:
224                 continue
225             if not isinstance(function_symbol, FunctionSymbol):
226                 continue
227             inferred_params = self._infer_param_types_from_body(
228                 state.node)
229             function_symbol.parameters = inferred_params
230             self._check_function_body(state, function_symbol)
231
232     def _infer_param_types_from_body(self, node: FunctionDef) -> list[
233         tuple[str, LanguageType]]:
234         """Infer parameter types by scanning the body for expression
235             contexts.
236
237             Any parameter used as an operand of a float operator (+. -. *.
238             /.) is
239             inferred as Float; a parameter used with an integer operator (+
240             - * /
241             == !=) is inferred as Integer. Parameters with no type hint
242             default to
243             Integer.
244         """
245         param_names = set(node.params)
246         inferred: dict[str, LanguageType] = {}
247
248     def scan_expr(n: ASTNode) -> None:
249         if isinstance(n, BinaryOp):
250             if n.op in {"+", "-", "*", "/", "==", "!="}:
251                 for operand in (n.left, n.right):
252                     if isinstance(operand, Identifier) and operand.
253                         name in param_names:
254                         inferred.setdefault(operand.name,
255                             LanguageType.INTEGER)
256             elif n.op in {"+.", "-.", "*.", "/."}:
257                 for operand in (n.left, n.right):
258                     if isinstance(operand, Identifier) and operand.
259                         name in param_names:
260                         inferred.setdefault(operand.name,
261                             LanguageType.FLOAT)
262             scan_expr(n.left)
263             scan_expr(n.right)
264         elif isinstance(n, FunctionCall):
265             for arg in n.args:
266                 scan_expr(arg)
267
268     def scan_stmt(n: ASTNode) -> None:
269         if isinstance(n, Assign):

```

```

259         scan_expr(n.value)
260     elif isinstance(n, Return):
261         scan_expr(n.expr)
262     elif isinstance(n, If):
263         scan_expr(n.condition)
264         scan_block(n.then_block)
265         if n.else_block:
266             scan_block(n.else_block)
267     elif isinstance(n, While):
268         scan_expr(n.condition)
269         scan_block(n.body)
270     elif isinstance(n, Block):
271         scan_block(n)
272     elif isinstance(n, FunctionCall):
273         for arg in n.args:
274             scan_expr(arg)
275
276 def scan_block(block: Block) -> None:
277     for stmt in block.statements:
278         scan_stmt(stmt)
279
280     scan_block(node.body)
281     return [(param, inferred.get(param, LanguageType.INTEGER)) for
282             param in node.params]
283
284 def _assign_variable_type(self, scope: SymbolTable, node: Assign,
285 value_type: LanguageType) -> None:
286     if value_type == LanguageType.VOID:
287         raise TypeCheckError(self._error_at(node, f"Cannot assign
288 the result of a void function to '{node.name}'"))
289     try:
290         symbol = scope.lookup(node.name)
291     except SymbolTableError:
292         scope.define_variable(node.name, value_type, initialized=
293 True)
294     return
295
296 if not isinstance(symbol, VariableSymbol):
297     raise TypeCheckError(self._error_at(node, f"'{node.name}'
298 is not a variable"))
299 if symbol.symbol_type != value_type:
300     raise TypeCheckError(
301 self._error_at(
302 node,
303 f"Cannot assign {value_type.value} to variable '{
304 node.name}' of type {symbol.symbol_type.value}",
305 )
306 )
307 symbol.initialized = True
308
309 def _infer_binary_type(
310 self,
311 node: BinaryOp,

```

```

306     left_type: LanguageType,
307     right_type: LanguageType,
308 ) -> LanguageType:
309     # Integer operators: both operands must be Integer
310     if node.op in {"+", "-", "*"}:
311         if left_type != LanguageType.INTEGER or right_type !=
           LanguageType.INTEGER:
312             raise TypeCheckError(
313                 self._error_at(node, f"Operator '{node.op}'
           requires two Integer operands")
314             )
315         return LanguageType.INTEGER
316
317     # Integer division: both operands must be Integer, result is
           Integer
318     if node.op == "/":
319         if left_type != LanguageType.INTEGER or right_type !=
           LanguageType.INTEGER:
320             raise TypeCheckError(
321                 self._error_at(node, f"Operator '{node.op}'
           requires two Integer operands")
322             )
323         return LanguageType.INTEGER
324
325     # Float operators: both operands must be Float
326     if node.op in {"+.", "-.", "*.", "/."}:
327         if left_type != LanguageType.FLOAT or right_type !=
           LanguageType.FLOAT:
328             raise TypeCheckError(
329                 self._error_at(node, f"Operator '{node.op}'
           requires two Float operands")
330             )
331         return LanguageType.FLOAT
332
333     if node.op in {"==", "!="}:
334         if not self._is_numeric(left_type) or not self._is_numeric(
           right_type):
335             raise TypeCheckError(
336                 self._error_at(
337                     node,
338                     f"Operator '{node.op}' requires two arithmetic
           operands",
339                 )
340             )
341         if left_type != right_type:
342             raise TypeCheckError(
343                 self._error_at(
344                     node,
345                     f"Operator '{node.op}' requires operands of the
           same type, got {left_type.value} and {
           right_type.value}",
346                 )
347             )

```

```

348         return LanguageType.BOOLEAN
349
350     raise TypeCheckError(self._error_at(node, f"Unsupported
351         operator: {node.op}"))
352
353 def _lookup_function_symbol(self, node: FunctionCall) ->
354     FunctionSymbol:
355     try:
356         symbol = self.global_scope.lookup(node.name)
357     except SymbolTableError as error:
358         raise TypeCheckError(self._error_at(node, str(error))) from
359         error
360     if not isinstance(symbol, FunctionSymbol):
361         raise TypeCheckError(self._error_at(node, f"'{node.name}',
362             is not a function"))
363     return symbol
364
365 @staticmethod
366 def _is_numeric(language_type: LanguageType) -> bool:
367     return language_type in {LanguageType.INTEGER, LanguageType.
368         FLOAT}
369
370 @staticmethod
371 def _literal_type(node: Literal) -> LanguageType:
372     if isinstance(node.value, bool):
373         return LanguageType.BOOLEAN
374     if isinstance(node.value, int):
375         return LanguageType.INTEGER
376     if isinstance(node.value, float):
377         return LanguageType.FLOAT
378     if isinstance(node.value, str):
379         return LanguageType.STRING
380     raise TypeCheckError(f"Unsupported literal value: {node.value!r}
381         ")
382
383 @staticmethod
384 def _error_at(node: ASTNode, message: str) -> str:
385     return f"[line {node.line}, col {node.column}] TypeError: {
386         message}"

```

Listing 8.10: components/type_checker.py — static type checker with inference

```

1 from __future__ import annotations
2
3 from dataclasses import dataclass
4 from typing import Callable
5
6 from components.ast_nodes import (
7     ASTNode,
8     Assign,
9     BinaryOp,
10    Block,
11    FunctionCall,

```

```

12     FunctionDef,
13     Identifier,
14     If,
15     Literal,
16     Program,
17     Return,
18     While,
19 )
20 from components.symbol_table import LanguageType
21
22
23 class InterpreterError(Exception):
24     pass
25
26
27 class _ReturnSignal(Exception):
28     def __init__(self, value: object) -> None:
29         self.value = value
30
31
32 @dataclass
33 class _FunctionRuntime:
34     node: FunctionDef
35
36
37 class _Environment:
38     def __init__(self, parent: _Environment | None = None) -> None:
39         self.parent = parent
40         self.values: dict[str, object] = {}
41
42     def define(self, name: str, value: object) -> None:
43         self.values[name] = value
44
45     def get(self, name: str) -> object:
46         if name in self.values:
47             return self.values[name]
48         if self.parent is not None:
49             return self.parent.get(name)
50         raise InterpreterError(f"Undefined variable: {name}")
51
52     def assign(self, name: str, value: object) -> None:
53         if name in self.values:
54             self.values[name] = value
55             return
56         if self.parent is not None and self.parent.contains(name):
57             self.parent.assign(name, value)
58             return
59         self.values[name] = value
60
61     def contains(self, name: str) -> bool:
62         if name in self.values:
63             return True
64         if self.parent is not None:

```

```

65         return self.parent.contains(name)
66     return False
67
68
69 class Interpreter:
70     _BUILTIN_PRINT = "print"
71
72     def __init__(self, output_callback: Callable[[str], None] | None =
None) -> None:
73         self._functions: dict[str, _FunctionRuntime] = {}
74         self._output_callback = output_callback or print
75
76     def execute(self, program: Program) -> None:
77         environment = _Environment()
78         for statement in program.statements:
79             if isinstance(statement, FunctionDef):
80                 self._functions[statement.name] = _FunctionRuntime(node
=statement)
81
82         for statement in program.statements:
83             if isinstance(statement, FunctionDef):
84                 continue
85             self._execute_statement(statement, environment)
86
87     def _execute_statement(self, node: ASTNode, environment:
_Environment) -> None:
88         if isinstance(node, Assign):
89             value = self._evaluate(node.value, environment)
90             environment.assign(node.name, value)
91             return
92
93         if isinstance(node, Return):
94             raise _ReturnSignal(self._evaluate(node.expr, environment))
95
96         if isinstance(node, If):
97             condition = self._evaluate(node.condition, environment)
98             if not isinstance(condition, bool):
99                 raise InterpreterError(self._error_at(node.condition, "
If condition must evaluate to Boolean"))
100             branch = node.then_block if condition else node.else_block
101             if branch is not None:
102                 self._execute_block(branch, _Environment(parent=
environment))
103             return
104
105         if isinstance(node, While):
106             while True:
107                 condition = self._evaluate(node.condition, environment)
108                 if not isinstance(condition, bool):
109                     raise InterpreterError(self._error_at(node.
condition, "While condition must evaluate to
Boolean"))
110                 if not condition:

```

```

111         break
112         self._execute_block(node.body, _Environment(parent=
            environment))
113     return
114
115     if isinstance(node, Block):
116         self._execute_block(node, _Environment(parent=environment))
117         return
118
119     if isinstance(node, FunctionCall):
120         self._evaluate(node, environment)
121         return
122
123     raise InterpreterError(self._error_at(node, f"Unsupported
        statement node: {type(node).__name__}"))
124
125 def _execute_block(self, block: Block, environment: _Environment)
    -> None:
126     for statement in block.statements:
127         self._execute_statement(statement, environment)
128
129 def _evaluate(self, node: ASTNode, environment: _Environment) ->
    object:
130     if isinstance(node, Literal):
131         return node.value
132
133     if isinstance(node, Identifier):
134         return environment.get(node.name)
135
136     if isinstance(node, BinaryOp):
137         left_value = self._evaluate(node.left, environment)
138         right_value = self._evaluate(node.right, environment)
139         return self._evaluate_binary(node, left_value, right_value)
140
141     if isinstance(node, FunctionCall):
142         return self._call_function(node, environment)
143
144     raise InterpreterError(self._error_at(node, f"Unsupported
        expression node: {type(node).__name__}"))
145
146 def _call_function(self, node: FunctionCall, environment:
    _Environment) -> object:
147     if node.name == self._BUILTIN_PRINT:
148         return self._call_builtin_print(node, environment)
149
150     function_runtime = self._functions.get(node.name)
151     if function_runtime is None:
152         raise InterpreterError(self._error_at(node, f"Undefined
            function: {node.name}"))
153
154     if len(node.args) != len(function_runtime.node.params):
155         raise InterpreterError(
156             self._error_at(

```

```

157         node,
158         f"Function '{node.name}' expects {len(
            function_runtime.node.params)} argument(s), got
            {len(node.args)}",
159     )
160 )
161
162 call_environment = _Environment(parent=environment)
163 argument_values = [self._evaluate(argument, environment) for
    argument in node.args]
164 for parameter_name, argument_value in zip(function_runtime.node
    .params, argument_values, strict=False):
165     call_environment.define(parameter_name, argument_value)
166
167 try:
168     self._execute_block(function_runtime.node.body,
        call_environment)
169 except _ReturnSignal as signal:
170     return signal.value
171 return None
172
173 def _call_builtin_print(self, node: FunctionCall, environment:
    _Environment) -> None:
174     if len(node.args) != 1:
175         raise InterpreterError(
176             self._error_at(node, f"Built-in function 'print'
                expects 1 argument, got {len(node.args)}")
177         )
178     value = self._evaluate(node.args[0], environment)
179     self._output_callback(f"{self._format_value(value)} : {self.
        _runtime_type(value).value}")
180     return None
181
182 def _evaluate_binary(self, node: BinaryOp, left_value: object,
    right_value: object) -> object:
183     if node.op == "+":
184         return left_value + right_value
185     if node.op == "-":
186         return left_value - right_value
187     if node.op == "*":
188         return left_value * right_value
189     if node.op == "/":
190         if int(right_value) == 0:
191             raise InterpreterError(self._error_at(node, "Division
                by zero"))
192         return int(left_value) // int(right_value)
193     if node.op == "+.":
194         return float(left_value) + float(right_value)
195     if node.op == "-.":
196         return float(left_value) - float(right_value)
197     if node.op == ".*":
198         return float(left_value) * float(right_value)
199     if node.op == "/.":

```



```

200         if float(right_value) == 0.0:
201             raise InterpreterError(self._error_at(node, "Division
                by zero"))
202         return float(left_value) / float(right_value)
203     if node.op == "==":
204         return left_value == right_value
205     if node.op == "!=":
206         return left_value != right_value
207     raise InterpreterError(self._error_at(node, f"Unsupported
        operator: {node.op}"))
208
209     @staticmethod
210     def _format_value(value: object) -> str:
211         if isinstance(value, str):
212             return f'"{value}"'
213         if isinstance(value, bool):
214             return "true" if value else "false"
215         return str(value)
216
217     @staticmethod
218     def _runtime_type(value: object) -> LanguageType:
219         if isinstance(value, bool):
220             return LanguageType.BOOLEAN
221         if isinstance(value, int):
222             return LanguageType.INTEGER
223         if isinstance(value, float):
224             return LanguageType.FLOAT
225         if isinstance(value, str):
226             return LanguageType.STRING
227         return LanguageType.VOID
228
229     @staticmethod
230     def _error_at(node: ASTNode, message: str) -> str:
231         return f"[line {node.line}, col {node.column}] RuntimeError: {
            message}"

```

Listing 8.11: components/interpreter.py — tree-walking interpreter

```

1  from __future__ import annotations
2
3  from dataclasses import dataclass
4
5  from components.ast_printer import ASTPrinter
6  from components.interpreter import Interpreter
7  from components.lexica import Lexer
8  from components.parse_tree_printer import ParseTreePrinter
9  from components.parser import Parser
10 from components.symbol_table import SymbolTable
11 from components.tokens import Token
12 from components.type_checker import TypeChecker
13
14
15 @dataclass

```

```

16 class PipelineResult:
17     tokens: list[Token]
18     parse_tree_text: str
19     ast_text: str
20     checked_scope: SymbolTable
21     outputs: list[str]
22
23
24 def run_pipeline(source_code: str) -> PipelineResult:
25     tokens = Lexer(source_code).tokenize()
26     tree = Parser(tokens).parse()
27     parse_tree_text = ParseTreePrinter().print(tree)
28     ast_text = ASTPrinter().print(tree)
29     checked_scope = TypeChecker().check(tree)
30
31     outputs: list[str] = []
32     Interpreter(output_callback=outputs.append).execute(tree)
33
34     return PipelineResult(
35         tokens=tokens,
36         parse_tree_text=parse_tree_text,
37         ast_text=ast_text,
38         checked_scope=checked_scope,
39         outputs=outputs,
40     )
41
42
43 def format_tokens(tokens: list[Token]) -> str:
44     lines = []
45     for token in tokens:
46         lines.append(
47             f"    {token.token_type.name:<14} {token.lexeme!r:<12} line={
48                 token.line} col={token.column}"
49         )
50     return "\n".join(lines)
51
52 def format_stage_output(result: PipelineResult) -> str:
53     sections = [
54         "=" * 60,
55         "    STAGE 1 -- TOKENS",
56         "=" * 60,
57         format_tokens(result.tokens),
58         "",
59         "=" * 60,
60         "    STAGE 2 -- PARSE TREE",
61         "=" * 60,
62         result.parse_tree_text,
63         "",
64         "=" * 60,
65         "    STAGE 3 -- AST",
66         "=" * 60,
67         result.ast_text,

```

```

68         "",
69         "=" * 60,
70         "    STAGE 4 -- TYPE CHECK",
71         "=" * 60,
72         result.checked_scope.format_table(),
73         "",
74         "=" * 60,
75         "    STAGE 5 -- EXECUTION",
76         "=" * 60,
77         "\n".join(result.outputs) if result.outputs else "(no output)",
78     ]
79     return "\n".join(sections)

```

Listing 8.12: components/pipeline.py — shared pipeline (CLI, UI, and tests)

A.3 Test Suite

```

1  from __future__ import annotations
2
3  import unittest
4
5  from components.interpreter import InterpreterError
6  from components.lexica import Lexer, LexerError
7  from components.parser import Parser, ParseError
8  from components.pipeline import run_pipeline
9  from components.symbol_table import LanguageType, SymbolTable,
10     SymbolTableError
11  from components.tokens import TokenType
12  from components.type_checker import TypeCheckError
13
14  #
15  # -----
16  #
17  # Lexer
18  # -----
19
20  class LexerTests(unittest.TestCase):
21      """Tests for components/lexica.py -- lexical analysis and
22      tokenisation."""
23
24      def test_integer_literal_token(self) -> None:
25          tokens = Lexer("42").tokenize()
26          self.assertEqual(tokens[0].token_type, TokenType.INT_LITERAL)
27          self.assertEqual(tokens[0].lexeme, "42")
28
29      def test_float_literal_token(self) -> None:
30          tokens = Lexer("3.14").tokenize()
31          self.assertEqual(tokens[0].token_type, TokenType.FLOAT_LITERAL)
32          self.assertEqual(tokens[0].lexeme, "3.14")

```

```

30
31 def test_bool_literals_produce_bool_literal_tokens(self) -> None:
32     tokens = Lexer("true false").tokenize()
33     self.assertEqual(tokens[0].token_type, TokenType.BOOL_LITERAL)
34     self.assertEqual(tokens[0].lexeme, "true")
35     self.assertEqual(tokens[1].token_type, TokenType.BOOL_LITERAL)
36     self.assertEqual(tokens[1].lexeme, "false")
37
38 def test_keywords_produce_correct_token_types(self) -> None:
39     tokens = Lexer("if else while def return").tokenize()
40     expected = [
41         TokenType.IF, TokenType.ELSE, TokenType.WHILE, TokenType.
42             DEF, TokenType.RETURN,
43     ]
44     actual = [t.token_type for t in tokens if t.token_type !=
45         TokenType.EOF]
46     self.assertEqual(actual, expected)
47
48 def test_identifier_not_confused_with_keyword(self) -> None:
49     tokens = Lexer("iffy whileloop print").tokenize()
50     self.assertEqual(tokens[0].token_type, TokenType.IDENTIFIER)
51     self.assertEqual(tokens[1].token_type, TokenType.IDENTIFIER)
52     self.assertEqual(tokens[2].token_type, TokenType.IDENTIFIER)
53
54 def test_two_character_operators(self) -> None:
55     tokens = Lexer("== !=").tokenize()
56     self.assertEqual(tokens[0].token_type, TokenType.EQUAL_EQUAL)
57     self.assertEqual(tokens[1].token_type, TokenType.BANG_EQUAL)
58
59 def test_float_dot_operators_tokenised(self) -> None:
60     tokens = Lexer("+. -. *. /.").tokenize()
61     expected = [TokenType.PLUS_DOT, TokenType.MINUS_DOT, TokenType.
62         STAR_DOT, TokenType.SLASH_DOT]
63     actual = [t.token_type for t in tokens if t.token_type !=
64         TokenType.EOF]
65     self.assertEqual(actual, expected)
66
67 def test_float_literal_not_confused_with_dot_operator(self) -> None
68 :
69     # "3.14" must produce FLOAT_LITERAL, not INT_LITERAL followed
70     by something
71     tokens = Lexer("3.14").tokenize()
72     self.assertEqual(tokens[0].token_type, TokenType.FLOAT_LITERAL)
73     self.assertEqual(tokens[0].lexeme, "3.14")
74
75 def test_source_position_tracked_across_lines(self) -> None:
76     tokens = Lexer("x\ny").tokenize()
77     self.assertEqual(tokens[0].line, 1)
78     self.assertEqual(tokens[1].line, 2)
79
80 def test_unexpected_character_raises_lexer_error(self) -> None:
81     with self.assertRaises(LexerError):
82         Lexer("@").tokenize()

```

```

77
78     def test_lone_exclamation_raises_lexer_error(self) -> None:
79         with self.assertRaises(LexerError):
80             Lexer("!5").tokenize()
81
82     def test_terminated_string_raises_lexer_error(self) -> None:
83         with self.assertRaises(LexerError):
84             Lexer('"hello"').tokenize()
85
86
87 #
88 # -----
89 # Symbol table
90 # -----
91
92 class SymbolTableTests(unittest.TestCase):
93     """Tests for components/symbol_table.py -- scoped symbol table and
94     semantic types."""
95
96     def test_define_variable_and_lookup(self) -> None:
97         table = SymbolTable()
98         table.define_variable("x", LanguageType.INTEGER, initialized=
99             True)
100         symbol = table.lookup("x")
101         self.assertEqual(symbol.symbol_type, LanguageType.INTEGER)
102
103     def test_duplicate_variable_definition_raises_error(self) -> None:
104         table = SymbolTable()
105         table.define_variable("x", LanguageType.INTEGER)
106         with self.assertRaises(SymbolTableError):
107             table.define_variable("x", LanguageType.FLOAT)
108
109     def test_duplicate_function_definition_raises_error(self) -> None:
110         table = SymbolTable()
111         table.define_function("f", LanguageType.INTEGER, [])
112         with self.assertRaises(SymbolTableError):
113             table.define_function("f", LanguageType.FLOAT, [])
114
115     def test_lookup_in_parent_scope(self) -> None:
116         parent = SymbolTable()
117         parent.define_variable("x", LanguageType.INTEGER, initialized=
118             True)
119         child = parent.child_scope()
120         symbol = child.lookup("x")
121         self.assertEqual(symbol.symbol_type, LanguageType.INTEGER)
122
123     def test_undefined_symbol_raises_error(self) -> None:
124         table = SymbolTable()
125         with self.assertRaises(SymbolTableError):
126             table.lookup("z")

```

```

123
124 def test_format_table_contains_variable_row(self) -> None:
125     table = SymbolTable()
126     table.define_variable("counter", LanguageType.INTEGER,
127                           initialized=True)
128     output = table.format_table()
129     self.assertIn("counter", output)
130     self.assertIn("variable", output)
131     self.assertIn("Integer", output)
132
133 def test_format_table_contains_function_row(self) -> None:
134     table = SymbolTable()
135     table.define_function("add", LanguageType.INTEGER, [("a",
136                                                         LanguageType.INTEGER)])
137     output = table.format_table()
138     self.assertIn("add", output)
139     self.assertIn("function", output)
140
141 # -----
142 # Parser
143 # -----
144
145 class ParserTests(unittest.TestCase):
146     """Tests for components/parser.py -- recursive-descent parsing."""
147
148     def _parse(self, source: str):
149         return Parser(Lexer(source).tokenize()).parse()
150
151     def test_missing_semicolon_raises_parse_error(self) -> None:
152         with self.assertRaises(ParseError):
153             self._parse("x = 5")
154
155     def test_unclosed_brace_raises_parse_error(self) -> None:
156         with self.assertRaises(ParseError):
157             self._parse("if (true) { x = 1;")
158
159     def test_operator_precedence_mul_before_add(self) -> None:
160         # 2 + 3 * 4 must be 14, not 20
161         result = run_pipeline("x = 2 + 3 * 4; print(x);")
162         self.assertEqual(result.outputs, ["14 : Integer"])
163
164     def test_operator_left_associativity(self) -> None:
165         # 10 - 3 - 2 must be 5 (left-assoc), not 9
166         result = run_pipeline("x = 10 - 3 - 2; print(x);")
167         self.assertEqual(result.outputs, ["5 : Integer"])
168
169     def test_parentheses_override_precedence(self) -> None:
170         # (2 + 3) * 4 must be 20

```

```

170     result = run_pipeline("x = (2 + 3) * 4; print(x);")
171     self.assertEqual(result.outputs, ["20 : Integer"])
172
173     def test_float_operator_precedence_mul_before_add(self) -> None:
174         # 1.0 +. 2.0 *. 3.0 must be 7.0 (*. binds tighter than +.)
175         result = run_pipeline("x = 1.0 +. 2.0 *. 3.0; print(x);")
176         self.assertEqual(result.outputs, ["7.0 : Float"])
177
178     def test_pipeline_exposes_parse_tree_and_ast(self) -> None:
179         result = run_pipeline("x = 5; print(x);")
180         self.assertIn("program", result.parse_tree_text)
181         self.assertIn("assignment", result.parse_tree_text)
182         self.assertIn("additive", result.parse_tree_text)
183         self.assertIn("term", result.parse_tree_text)
184         self.assertIn("factor", result.parse_tree_text)
185         self.assertIn("function_call", result.parse_tree_text)
186         self.assertIn("Program", result.ast_text)
187         self.assertIn("Assign", result.ast_text)
188         self.assertIn("FunctionCall 'print'", result.ast_text)
189
190
191     #
192     -----
193
194     # Type checker
195     #
196     -----
197
198
199     class TypeCheckerTests(unittest.TestCase):
200         """Tests for components/type_checker.py -- static type inference and
201         checking."""
202
203         def test_integer_arithmetic_stays_integer(self) -> None:
204             result = run_pipeline("x = 3 + 4; print(x);")
205             self.assertEqual(result.outputs, ["7 : Integer"])
206
207         def test_division_always_produces_integer(self) -> None:
208             # integer / integer -> Integer (no implicit float conversion)
209             result = run_pipeline("x = 10 / 2; print(x);")
210             self.assertEqual(result.outputs, ["5 : Integer"])
211
212         def test_float_dot_operators_produce_float(self) -> None:
213             result = run_pipeline("x = 1.0 +. 2.5; print(x);")
214             self.assertEqual(result.outputs, ["3.5 : Float"])
215
216         def test_float_subtraction_produces_float(self) -> None:
217             result = run_pipeline("x = 5.0 -. 1.5; print(x);")
218             self.assertEqual(result.outputs, ["3.5 : Float"])
219
220         def test_float_multiplication_produces_float(self) -> None:
221             result = run_pipeline("x = 2.0 *. 3.0; print(x);")
222             self.assertEqual(result.outputs, ["6.0 : Float"])

```

```

218
219 def test_float_division_produces_float(self) -> None:
220     result = run_pipeline("x = 9.0 /. 4.0; print(x);")
221     self.assertEqual(result.outputs, ["2.25 : Float"])
222
223 def test_integer_with_float_op_raises_type_error(self) -> None:
224     # +. requires Float operands; integers must not be accepted
225     with self.assertRaises(TypeError):
226         run_pipeline("x = 2 +. 3; print(x);")
227
228 def test_mixed_int_float_addition_raises_type_error(self) -> None:
229     # Integer + Float is no longer valid; must use +.
230     with self.assertRaises(TypeError):
231         run_pipeline("x = 1 + 2.5; print(x);")
232
233 def test_string_variable_type_inferred(self) -> None:
234     result = run_pipeline('x = "hello"; print(x);')
235     self.assertEqual(result.outputs, ['"hello" : String'])
236
237 def test_boolean_literal_type_inferred(self) -> None:
238     result = run_pipeline("x = true; print(x);")
239     self.assertEqual(result.outputs, ["true : Boolean"])
240
241 def test_boolean_equality_raises_type_error(self) -> None:
242     # Boolean == Boolean is not allowed; == only accepts arithmetic
243     operands
244     with self.assertRaises(TypeError):
245         run_pipeline('x = true; if (x == false) { print("then"); }'
246                     'else { print("else"); }')
247
248 def test_mixed_int_float_equality_raises_type_error(self) -> None:
249     # == and != must not accept mixed Integer/Float (no overloading
250     )
251     with self.assertRaises(TypeError):
252         run_pipeline("x = 5; y = 3.14; if (x == y) { print(x); }")
253
254 def test_mixed_int_float_inequality_raises_type_error(self) -> None:
255     :
256     with self.assertRaises(TypeError):
257         run_pipeline("x = 5; y = 3.14; if (x != y) { print(x); }")
258
259 def test_arithmetic_on_string_raises_type_error(self) -> None:
260     with self.assertRaises(TypeError):
261         run_pipeline('x = "a" + 1;')
262
263 def test_if_non_boolean_condition_raises_type_error(self) -> None:
264     with self.assertRaises(TypeError):
265         run_pipeline("if (1) { x = 2; }")
266
267 def test_while_non_boolean_condition_raises_type_error(self) ->
None:
268     with self.assertRaises(TypeError):
269         run_pipeline("x = 0; while (x) { x = x + 1; }")

```



```

266
267 def test_assignment_type_mismatch_raises_type_error(self) -> None:
268     with self.assertRaises(TypeError):
269         run_pipeline('x = 5; x = "hello";')
270
271 def test_undefined_variable_raises_type_error(self) -> None:
272     with self.assertRaises(TypeError):
273         run_pipeline("print(z);")
274
275 def test_function_wrong_arg_count_raises_type_error(self) -> None:
276     with self.assertRaises(TypeError):
277         run_pipeline("def f(a) { return a; } f(1, 2);")
278
279 def test_function_arg_type_mismatch_raises_type_error(self) -> None
280 :
281     # first call infers a: Integer; second call with String must
282     fail
283     with self.assertRaises(TypeError):
284         run_pipeline('def f(a) { return a; } f(1); f("hello");')
285
286 def test_uncalled_function_body_type_error_is_caught(self) -> None:
287     # type error inside body must be detected even if function is
288     never called
289     with self.assertRaises(TypeError):
290         run_pipeline('def bad() { y = "hello" + 1; }')
291
292 def test_valid_uncalled_function_does_not_raise(self) -> None:
293     # a well-typed but uncalled function should not raise
294     result = run_pipeline("def add(a, b) { return a + b; }")
295     self.assertEqual(result.outputs, [])
296
297 #
298
299 -----
300
301 # Integration (full pipeline)
302 #
303 -----
304
305 class IntegrationTests(unittest.TestCase):
306     """End-to-end tests through all four pipeline stages."""
307
308     # --- Unary minus ---
309
310     def test_unary_minus_integer(self) -> None:
311         result = run_pipeline("x = -5; print(x);")
312         self.assertEqual(result.outputs, ["-5 : Integer"])
313
314     def test_unary_minus_float(self) -> None:
315         result = run_pipeline("y = -. 3.14; print(y);")
316         self.assertEqual(result.outputs, ["-3.14 : Float"])

```

```

312 def test_unary_minus_dot_variable(self) -> None:
313     result = run_pipeline("x = 1.5; y = -. x; print(y);")
314     self.assertEqual(result.outputs, ["-1.5 : Float"])
315
316 def test_unary_minus_variable(self) -> None:
317     result = run_pipeline("x = 10; y = -x; print(y);")
318     self.assertEqual(result.outputs, ["-10 : Integer"])
319
320 def test_unary_minus_inside_expression(self) -> None:
321     result = run_pipeline("x = 3 + -2; print(x);")
322     self.assertEqual(result.outputs, ["1 : Integer"])
323
324 # --- Control flow ---
325
326 def test_if_then_branch_executes(self) -> None:
327     result = run_pipeline(
328         'x = 5; if (x == 5) { print("yes"); } else { print("no"); }
329     )
330     self.assertEqual(result.outputs, ['"yes" : String'])
331
332 def test_if_else_branch_executes_when_condition_false(self) -> None:
333     result = run_pipeline(
334         'x = 0; if (x == 5) { print("yes"); } else { print("no"); }
335     )
336     self.assertEqual(result.outputs, ['"no" : String'])
337
338 def test_while_loop_executes_repeatedly(self) -> None:
339     result = run_pipeline("x = 3; while (x != 0) { print(x); x = x
340         - 1; }")
341     self.assertEqual(result.outputs, ["3 : Integer", "2 : Integer",
342         "1 : Integer"])
343
344 # --- Division by zero ---
345
346 def test_integer_division_by_zero_raises_error(self) -> None:
347     with self.assertRaises(InterpreterError):
348         run_pipeline("x = 10 / 0; print(x);")
349
350 def test_float_division_by_zero_raises_error(self) -> None:
351     with self.assertRaises(InterpreterError):
352         run_pipeline("x = 10.0 /. 0.0; print(x);")
353
354 # --- Functions ---
355
356 def test_function_integer_return(self) -> None:
357     result = run_pipeline("def square(n) { return n * n; } print(
358         square(7));")
359     self.assertEqual(result.outputs, ["49 : Integer"])
360
361 def test_function_type_inference(self) -> None:

```

```

359     result = run_pipeline(
360         "def add(a, b) { return a +. b; } result = add(2.0, 3.5);
          print(result);"
361     )
362     self.assertEqual(result.outputs, ["5.5 : Float"])
363     self.assertIn("result      variable   Float", result.
          checked_scope.format_table())
364
365     def test_function_value_parameter_passing(self) -> None:
366         # Modifying a parameter inside a function must not affect the
          caller's variable
367         result = run_pipeline(
368             "def inc(a) { a = a + 1; return a; } x = 10; y = inc(x);
              print(x); print(y);"
369         )
370         self.assertEqual(result.outputs, ["10 : Integer", "11 : Integer
          "])
371
372     def test_nested_function_calls(self) -> None:
373         result = run_pipeline(
374             "def add(a, b) { return a + b; } print(add(add(1, 2), 3));"
375         )
376         self.assertEqual(result.outputs, ["6 : Integer"])
377
378         # --- print() with all four types ---
379
380     def test_print_integer(self) -> None:
381         result = run_pipeline("print(42);")
382         self.assertEqual(result.outputs, ["42 : Integer"])
383
384     def test_print_float(self) -> None:
385         result = run_pipeline("print(3.14);")
386         self.assertEqual(result.outputs, ["3.14 : Float"])
387
388     def test_print_boolean(self) -> None:
389         result = run_pipeline("print(true);")
390         self.assertEqual(result.outputs, ["true : Boolean"])
391
392     def test_print_string(self) -> None:
393         result = run_pipeline('print("plc");')
394         self.assertEqual(result.outputs, ['"plc" : String'])
395
396         # --- Boolean expressions as values ---
397
398     def test_comparison_result_assigned_to_variable(self) -> None:
399         result = run_pipeline("x = 5; y = 5; flag = (x == y); print(
          flag);")
400         self.assertEqual(result.outputs, ["true : Boolean"])
401
402     def test_comparison_result_printed_directly(self) -> None:
403         result = run_pipeline("x = 10; print(x != 0);")
404         self.assertEqual(result.outputs, ["true : Boolean"])
405

```

```

406 def test_comparison_result_false(self) -> None:
407     result = run_pipeline("a = 1; b = 2; eq = (a == b); print(eq);"
408     )
409     self.assertEqual(result.outputs, ["false : Boolean"])
410
411     # --- Reassignment ---
412
413 def test_variable_can_be_reassigned_same_type(self) -> None:
414     result = run_pipeline("x = 1; x = 2; x = 3; print(x);")
415     self.assertEqual(result.outputs, ["3 : Integer"])
416
417 def test_scope_isolation_if_block(self) -> None:
418     # Variable defined inside an if-block must not be visible
419     outside it
420     with self.assertRaises(TypeError):
421         run_pipeline(
422             "x = 1; if (x == 1) { inner = 99; } print(inner);"
423         )
424
425 def test_string_equality_raises_type_error(self) -> None:
426     # == is only valid between arithmetic (Integer/Float) operands
427     with self.assertRaises(TypeError):
428         run_pipeline('a = "hello"; b = "world"; if (a == b) { print'
429             '(a); }')
430
431 def test_integer_floor_division(self) -> None:
432     # 7 / 2 must be 3 (floor), not 3.5
433     result = run_pipeline("x = 7 / 2; print(x);")
434     self.assertEqual(result.outputs, ["3 : Integer"])
435
436 def test_void_function_assignment_raises_type_error(self) -> None:
437     # Assigning the result of a void function (no return) must be a
438     type error
439     with self.assertRaises(TypeError):
440         run_pipeline("def greet() { print(42); } x = greet();")
441
442 def test_builtin_print_assignment_raises_type_error(self) -> None:
443     with self.assertRaises(TypeError):
444         run_pipeline("x = print(42);")
445
446 if __name__ == "__main__":
447     unittest.main()

```

Listing 8.13: tests/test_pipeline.py — automated regression suite (75 tests)